

CONTROL EN IMPEDANCIA PARA LA ACTUACIÓN RÁPIDA DE ROBOTS CON VISIÓN BASADA EN EVENTOS

TRABAJO FIN DE GRADO
PARA LA TITULACIÓN DEL
GRADO EN ROBÓTICA

AUTOR:

ADRIÁN FERNÁNDEZ CASAS

TUTORES:

JUAN ANTONIO CORRALES RAMÓN
DPTO ELECTRÓNICA E COMPUTACIÓN - USC

CARLOS MANUEL MATEO AGULLÓ
LABORATOIRE ICB — UNIVERSITÉ
BOURGOGNE EUROPE



ESCOLA POLITÉCNICA SUPERIOR
DE ENXEÑARÍA

CONVOCATORIA:
FEBRERO DE 2026

UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

TRABAJO FIN DE GRADO

Control en impedancia para la actuación rápida de robots con visión basada en eventos

AUTOR:

Adrián Fernández Casas

TUTORES:

Juan Antonio Corrales Ramón

Carlos Manuel Mateo Agulló

*Este trabajo se presenta para cumplir con los requerimientos normativos
exigidos para alcanzar el Grado en Robótica
en la*

Escola Politécnica Superior de Enxeñaría.

Fecha de la Convocatoria: Febrero de 2026



ESCOLA POLITÉCNICA SUPERIOR
DE ENXEÑARÍA

Autorización

Yo, **Adrián Fernández Casas**, alumno de la titulación del Grado en Robótica de la **EPS de Enxeñaría** de la **Universidade de Santiago de Compostela**, con DNI 35624898Z, autorizo la ubicación de la siguiente copia de mi **Trabajo Fin de Grado** en la biblioteca de Intercentros para que pueda ser consultada por las personas que lo deseen.

Firmado:

Fecha: 30 de enero de 2026

*La ciencia no es más que perversión en sí misma,
a menos que tenga como objetivo último mejorar la humanidad.*

Nikola Tesla

Resumo

CONTROL EN IMPEDANCIA PARA A ACTUACIÓN RÁPIDA DE ROBOTS CON VISIÓN BASEADA EN EVENTOS

por
ADRIÁN FERNÁNDEZ CASAS

Palabras Chave: Impedancia, torques, dinámica, control visual, IBVS, eventos, robot colaborativo

Este traballo trata de controlar un robot colaborativo de maneira eficiente, cunha cámara de eventos montada *eye-in-hand*. O obxectivo do proxecto é realizar un control rápido do robot para que poda alcanzar un obxecto que teña unha velocidade considerable.

Por unha parte, implementárase un control en impedancia para un robot colaborativo, para lograr un control máis directo. Levarase a cabo unha tarefa de aprendizaxe por reforzo profundo, para obter os parámetros necesarios para o control de maneira dinámica. Procúrase que a selección dos parámetros sexa adaptativa ao movemento e á tarefa que leva a cabo o robot.

Por outra parte, implementárase unha tarefa de control visual baseado en IBVS para poder alcanzar o obxecto co robot. Se realizarán varias probas utilizando unha cámara RGBD, con diferentes métodos de detección. Despois, finalizarase o proxecto implementando o control visual ca información obtida a partir dunha cámara de eventos.

Os resultados obtidos demostran unha vía de implementación para o axente de aprendizaxe por reforzo, consolidando a viabilidade da proposta deste TFG. No que respecta ao control visual, demostrouse que o controlador foi implementado correctamente, cuns resultados que demostran que o robot é capaz de situarse enriba do obxecto en movemento e seguilo, corrixindo a altura desexada e a orientación.

Resumen

CONTROL EN IMPEDANCIA PARA LA ACTUACIÓN RÁPIDA DE ROBOTS CON VISIÓN BASADA EN EVENTOS

por
ADRIÁN FERNÁNDEZ CASAS

Palabras Clave: Impedancia, torques, dinámica, control visual, IBVS, eventos, robot colaborativo

Este trabajo trata de controlar un robot colaborativo de manera eficiente, con una cámara de eventos montada *eye-in-hand*. El objetivo del proyecto es realizar un control rápido del robot para que pueda alcanzar un objeto que tenga una velocidad considerable.

Por una parte, se implementará un control en impedancia para un robot colaborativo, para lograr un control más directo. Se llevará a cabo una tarea de aprendizaje por refuerzo profundo para obtener los parámetros necesarios para el control de manera dinámica. Se busca que la selección de los parámetros sea adaptativa al movimiento y a la tarea que lleva a cabo el robot.

Por otra parte, se implementará una tarea de control visual basado en IBVS para poder alcanzar el objeto con el robot. Se realizarán varias pruebas utilizando una cámara RGBD, con diferentes métodos de detección. Después, se finalizará el proyecto implementando el control visual con la información obtenida a partir de una cámara de eventos.

Los resultados obtenidos demuestran una vía de implementación para el agente de aprendizaje por refuerzo, consolidando la viabilidad de la propuesta de este TFG. En lo que respecta al control visual, se ha demostrado que el controlador ha sido implementado correctamente, con unos resultados que demuestran que el robot es capaz de situarse encima del objeto en movimiento y seguirlo, corrigiendo la altura deseada y la orientación.

Abstract

IMPEDANCE CONTROL FOR RAPID ROBOT ACTUATION WITH EVENT-BASED VISION

by

ADRIÁN FERNÁNDEZ CASAS

Keywords: Impedance, torque, dynamics, visual servoing, IBVS, events, collaborative robot

This work aims to control a collaborative robot efficiently, using an eye-in-hand mounted event camera. The objective of the project is to control the robot quickly, so that it can reach an object moving at considerable speed.

On the one hand, impedance control will be implemented for a collaborative robot to achieve more direct control. A deep reinforcement learning task will be carried out to obtain the parameters necessary for dynamic control. The aim is for the selection of parameters to be adaptive to the movement and task performed by the robot.

On the other hand, a visual control task based on IBVS will be implemented in order to reach the object with the robot. Several tests will be carried out using an RGBD camera, with different detection methods. The project will then be completed by implementing visual control with the information obtained from an event camera.

The results obtained demonstrate a way of implementing the reinforcement learning agent, consolidating the viability of this TFG proposal. With regard to visual control, it has been demonstrated that the controller has been implemented correctly, with results showing that the robot is capable of positioning itself above the moving object and following it, correcting the desired height and orientation.

Agradecementos

En todos estes anos de estudo sempre me sentín moi respaldado pola xente ao meu redor. Na miña casa sempre velaron pola miña educación, aínda cando non era dos que máis se esforzasen na clase. Mais por enriba de todo, o máis importante para a miña familia foi sempre ensinarme uns valores cos que puidese recorrer o mundo, orgulloso de quen son e de onde veño. Por eso mesmo, comezo estes agradecementos por eles, porque dende que nacín, sempre me coidaron e aconsellaron da mellor maneira posible. Por eso mesmo, moitas grazas a toda a miña familia. A miña madriña por sempre acordarse de min, aconsellarme e quererme. A meu padriño, por ser quen fixo que dende pequeno sempre me gustase a enxeñaría. A miña avoa, por coidarme sempre moito máis do que podería facer ninguén, e non regañarme ningunha vez en toda a miña vida. A meu avó, por ensinarme a crecer neste mundo, que nada me gustaría máis que poder velo unha vez máis, e poder ensinalle a persoa na que se está convertindo o seu neto. A miña irmá, por haber sido sempre un exemplo a seguir e, incomprendiblemente, non haberse fartado dun irmao tan pesado. Pero sobre todo, grazas a meus pais, por ser os grandes guías da miña vida, por quererme indefinidamente, como tan só un pai e unha nai poden, por todos os sacrificios que fixeron sempre por darme o mellor, por complicado que puidese ser. Grazas por querer sempre o mellor para min, pero escoitándome e intentando comprenderme. E grazas por sempre facerme soñar e mostrarme a quen me quero parecer no futuro.

Grazas tamén a Alba, por haberme apoiado tanto sempre, aínda cando vexo as cousas moi mal, sempre consigues animarme e alentarme a dar sempre o máximo de min. Grazas por ensinarme e mostrarme un amor que nunca imaxinei, por ser a miña compañeira en todos os momentos, verdadeiramente non te imaxinas todo o que me axudache, este TFG é tamén grazas a ti.

Grazas a todas as persoas cas que tiveron a sorte de encontrarme nestes anos de universidade. David, grazas por non só ser o meu compañeiro de piso, senón ser tamén un dos meus mellores amigos, e un gran regalo que me deu a vida universitaria. Gabriel (dígocho en galego, síntoi), grazas por axudarme, acompañarme en todo este traxecto, e ser un gran apoio, ti tamén fixeche posible este TFG. Manute, grazas por estar ahí, axudando sempre en todo o posible, sacarme un riso ata cando tiveron un mal día (exame de mates), e xa sabes, na universidade todos somos amigos.

E por último, moitas grazas a Carlos, por haberme acollido estes meses no seu laboratorio, por non só haber sido un supervisor, senón sobre todo un gran profesor, e facer que nestes 5 meses houbese aprendido máis que en 10 anos de universidade, e grazas tamén a Juan Antonio, por haber sido o meu titor e facer que puidese haber tido unha experiencia tan formidable e enriquecedora nas prácticas.

Índice

Autorización.....	iii
Resumo	v
Resumen	vi
Abstract	vii
Agradecimientos.....	viii
Índice de figuras	xi
Índice de Tablas.....	xii
Lista de Abreviaciones.....	xiii
Lista de Símbolos	xiii
1 Introducción	1
1.1 Contexto del proyecto	1
1.2 Justificación y pertinencia	1
2. Marco teórico.....	2
2.1. Definición del control en impedancia.....	2
2.1.1. Robótica manipuladora	2
2.1.2. Robótica colaborativa.....	4
2.2. Definición del control visual.....	4
2.2.1. Eye-in-hand vs eye-to-hand	4
2.3. Cámaras de eventos.....	5
2.4. Aprendizaje por refuerzo.....	6
2.4.1. Aprendizaje por refuerzo profundo (Deep RL)	6
3. Estado del arte	8
3.1. Control en impedancia	8
3.1.1. Problema actual del control en impedancia.....	8
3.1.2. Control de impedancia variable.....	8
3.1.3. Control de impedancia adaptativo.....	9
3.1.4. Control de impedancia variable mediante aprendizaje.....	9
3.1.5. Propuesta de innovación en el control de impedancia variable mediante aprendizaje	10
3.2. Control visual	10
3.2.1. Control visual clásico: IBVS y PBVS	10
3.2.2. Control visual híbrido (2.5D Visual Servoing)	11
3.2.3. Control visual fotométrico	12
3.2.4. Deep visual servoing	12
3.2.5. Control visual dinámico	12
4. Herramientas y recursos	13
4.1. Hardware	13
4.1.1. Robot	13
4.1.2. Cámaras.....	15
4.1.3. Ordenador	16
4.1.4. Entorno de trabajo	16
4.2. Software.....	17
4.2.1. Control del robot: libfranka	17
4.2.2. Simulador: Isaac Sim.....	17
4.2.3. Plataforma software ROS 2.....	18
4.2.4. Framework de RL: Stable Baselines 3.....	18
4.2.5. Control visual: VISP	19

5. Desarrollo.....	20
5.1. <i>Objetivos</i>	20
5.2. <i>Metodología</i>	20
5.2.1. Control del robot	20
5.2.2. Tarea de aprendizaje por refuerzo	20
5.2.3. Control visual	21
5.3. <i>Control del robot</i>	21
5.3.1. Control cinemático	21
5.3.2. Módulo de autocalibración.....	22
5.3.3. Control en impedancia	23
5.4. <i>Aprendizaje de los parámetros mediante RL</i>	24
5.4.1. Entorno de simulación en Isaac Sim.....	24
5.4.2. Elección del algoritmo: Soft Actor Critic.....	25
5.4.3. Implementación del agente	27
5.4.4. Comunicación entre el simulador y el agente	28
5.4.5. Desarrollo del aprendizaje	28
5.4.6. Problema con controlador y redefinición de la tarea	29
5.4.7. Nuevo enfoque: perfil trapezoidal	30
5.4.8. Control basado en aceleración	31
5.5. <i>Control visual</i>	31
5.5.1. Simulación	32
5.5.2. Control visual en el robot real: AprilTags	33
5.5.3. Control visual en robot real: Seguimiento de la pelota	34
5.5.4. Control visual con cámara de eventos	34
6. Resultados.....	36
6.1. <i>Control en impedancia</i>	36
6.1.1. Modificaciones en el agente	36
6.1.2. Variantes en la implementación	38
6.2. <i>Control visual</i>	44
6.2.1. Simulación	44
6.2.2. Robot real: AprilTags	45
6.2.3. Robot real: Pelota con cámara RGBD	45
6.2.4. Robot real: Pelota con cámara de eventos.....	46
7. Conclusiones	47
7.1. <i>Control en impedancia</i>	47
7.2. <i>Control visual</i>	47
7.3. <i>Reflexión final</i>	48
Anexo A: Links para vídeos y códigos	49
Referencias	51

Índice de figuras

Figura 2.1: Sistema masa-muelle-amortiguador, principio fundamental del control en impedancia	2
Figura 2.2: Cadena cinemática del robot Franka Research 3, un robot 7R.....	3
Figura 2.3: Escena con cámara <i>eye-in-hand</i> frente a <i>eye-to-hand</i>	5
Figura 2.4: Diferencia entre imagen con una cámara convencional y una cámara de eventos. Fuente: WEB de Prophesee [7]	6
Figura 3.1: Resultados de PBVS, IBVS y HVSC (híbrido). Fuente: Li et al., página 15 [22].....	11
Figura 4.1: Robot colaborativo Franka Research 3	13
Figura 4.2: Cámara RGBD Intel RealSense D405.....	15
Figura 4.3: Cámara de eventos Prophesee EVK4.....	15
Figura 4.4: Entorno de trabajo sin objetos	16
Figura 4.5: Entorno de trabajo con objetos.....	16
Figura 4.6: Simulador Isaac Sim.....	18
Figura 5.1: Entorno de trabajo para la autocalibración.....	23
Figura 5.2: Resultados de entrenamiento de diferentes algoritmos de Deep RL	26
Figura 5.3: Estructura del algoritmo de Deep RL Soft Actor Critic [41]	27
Figura 5.4: Franka Research 3 en pose de reposo (<i>home</i>)	29
Figura 5.5: Perfil trapezoidal.....	31
Figura 5.6: Diferentes tipos de AprilTags utilizados en robótica	33
Figura 6.1: <i>Critic loss</i> antes de la normalización	36
Figura 6.2: <i>Critic loss</i> después de la normalización.....	36
Figura 6.3: Explosión de la entropía: coeficiente de entropía	37
Figura 6.4: Explosión de la entropía: recompensa media del episodio.....	37
Figura 6.5: Estabilización de la entropía: coeficiente de entropía.....	37
Figura 6.6: Estabilización de la entropía: recompensa media del episodio	37
Figura 6.7: Longitud media del episodio	38
Figura 6.8: Recompensa media del episodio	38
Figura 6.9: Evolución del parámetro <i>count_D</i>	38
Figura 6.10: Evolución de la distancia a la meta para $K = 600$	39
Figura 6.11: Evolución del <i>jerk</i> para $K = 600$	39
Figura 6.12: Evolución velocidades articulares para $K = 600$	39
Figura 6.13: Evolución de la distancia a la meta para $K = 300$	40
Figura 6.14: Evolución del <i>jerk</i> para $K = 300$	40
Figura 6.15: Evolución de la distancia a la meta con ganancias generadas siguiendo un perfil trapezoidal	41
Figura 6.16: Evolución del <i>jerk</i> con ganancias generadas siguiendo un perfil trapezoidal.....	41
Figura 6.17: Evolución de las velocidades articulares con ganancias generadas siguiendo un perfil trapezoidal.....	42
Figura 6.18: Evolución de la distancia a la meta con aceleración constante y $K = 600$	42
Figura 6.19: Evolución del <i>jerk</i> con aceleración constante y $K = 600$	43
Figura 6.20: Evolución de las velocidades articulares con aceleración constante y $K = 600$	43
Figura 6.21: Evolución del <i>jerk</i> con aceleración constante y $K = 300$	44
Figura 6.22: Evolución de las velocidades articulares con aceleración constante y $K = 300$	44
Figura 6.23: Resultados del entrenamiento de YOLOv4	45

Índice de Tablas

Tabla 3.1: Comparación entre PBVS y IBVS.....	11
Tabla 4.1: Límites del espacio cartesiano del FR3 [27]	14
Tabla 4.2: Límites del espacio articular del FR3 [27]	14
Tabla 4.3: Especificaciones del sistema	16

Lista de Abreviaciones

TFG	Trabajo Fin de Grado
IBVS	Image-Based Visual Servoing
PBVS	Position-Based Visual Servoing
RL	Reinforcement Learning
dof	Degrees of liberty
fps	Frames por segundo
SB3	Stable Baselines 3

Lista de Símbolos

τ	Torque	$N \cdot m$
F	Fuerza	N
F	Frecuencia	Hz
P	Potencia	W
$M(q)$	Masa inercial articular	$Kg \cdot m^2$
$C(q, \dot{q})$	Coriolis articular	$Kg \cdot m^2/s$
$g(q)$	Gravedad articular	$N \cdot m$

Dedicado á miña familia ...

1 INTRODUCCIÓN

Este Trabajo Fin de Grado se ha desarrollado en el marco de un proyecto de investigación en robótica, llevado a cabo en el laboratorio ICB (Laboratoire Interdisciplinaire Carnot de Bourgogne), perteneciente a la Université de Bourgogne (UBE), en Dijon.

El presente trabajo se centra en los aspectos de desarrollo e investigación correspondientes a mi contribución personal dentro de dicho proyecto. Por este motivo, en esta introducción se presenta el contexto general del proyecto, se describe mi participación específica y se establece la justificación y pertinencia del trabajo, así como la delimitación del problema abordado y los objetivos.

1.1 Contexto del proyecto

El proyecto en el que se desarrolla este Trabajo Fin de Grado envuelve el uso de un brazo robótico colaborativo con una cámara de eventos montada *eye-in-hand*, que tendrá que ser capaz de detectar eficientemente un objeto en movimiento y situarse encima del mismo, teniendo una respuesta rápida. Lo que se busca con este proyecto es utilizar la tecnología de las cámaras de eventos, las cuales ofrecen una muy baja latencia, pero debido a su modo de funcionamiento, presentan una mayor dificultad de utilización.

Por otra parte, para aprovechar al máximo esta baja latencia, también es necesario realizar un control óptimo del robot, intentando construir un control rápido y eficiente, que, utilizando la información extraída de la cámara de eventos, sea capaz de situar al efector final del robot encima del objeto, haciendo que “siga” a dicho elemento. En el contexto futuro del proyecto, no solo tendrá que situarse encima del objeto, sino que también se llevará a cabo la manipulación del mismo.

Por ende, el proyecto se dividirá en 2 partes: control y visión. Este Trabajo Fin de Grado se desarrolla en el contexto del control del robot, abordando únicamente esta parte del proyecto, dejando de lado todo el proceso de obtención de la información con la cámara de eventos. Se hará el supuesto de que simplemente obtendremos las coordenadas en píxeles del centroide del objeto o una imagen con la segmentación de este. Aun así, será necesario entender el contexto del proyecto, ya que es imperativo entender el funcionamiento de las cámaras de eventos para adaptar correctamente el control del robot a este tipo de sensor perceptivo.

1.2 Justificación y pertinencia

La interacción física eficiente entre los robots y su entorno constituye un reto en la robótica moderna, especialmente si hablamos de tareas que necesiten de una gran rapidez en el control, al mismo tiempo que precisión. En este contexto, situamos este Trabajo Fin de Grado, donde no solo necesitamos estas características, sino que por la tipología del brazo robot y la tarea futura del proyecto se necesitará hacer un control dinámico. Este tipo de control sigue implicando hoy en día un desarrollo arduo y problemático, que conlleva la elección de diferentes parámetros necesarios en la ley de control. La elección de estos parámetros presenta limitaciones importantes, las cuáles esta propuesta pretende resolver.

Paralelamente, hay que añadir a la tarea la percepción visual del entorno, para adaptar el comportamiento del robot. El control visual proporciona esta adaptación, donde a partir de la información obtenida mediante la cámara, somos capaces de controlar al robot para que siga un objeto en concreto. Al tratarse de un control dinámico lo primero que podemos pensar es en utilizar control visual dinámico, pero en este trabajo se presenta una propuesta más sencilla, pero igualmente eficiente.

Asimismo, este Trabajo Fin de Grado se adecua a las competencias del grado en Robótica, al integrar los conocimientos adquiridos en: control y modelado de sistemas dinámicos, robótica manipuladora, aprendizaje por refuerzo, el uso de herramientas de simulación y comunicación entre procesos. El trabajo aborda un problema de complejidad acorde al nivel de un TFG en un contexto de investigación.

2. MARCO TEÓRICO

En la introducción se han comentado las tareas a realizar, pero es necesario desarrollarlas con mayor detalle y explicar qué son estos conceptos. Es necesario para entender qué se está haciendo realmente en este Trabajo Fin de Grado, qué es lo que se busca conseguir, y por qué vamos a utilizar una metodología en concreto. En este apartado se explicarán conceptos clave, que facilitarán la comprensión del desarrollo del trabajo.

2.1. Definición del control en impedancia

Como establece Hogan en 1985 [1], el control en impedancia es una estrategia de control robótico que relaciona la dinámica del robot con el movimiento deseado del mismo. Este tipo de estrategia no sólo nos permite controlar el movimiento del robot, sino que nos aporta información sobre la interacción con el entorno; si el robot interactúa con algo, sabemos cómo responde dinámicamente al contacto. La idea clave es que el brazo robótico se comporta como un sistema masa-muelle-amortiguador (ver **Figura 2.1**), un sistema muy común en ingeniería de control. ¿Cuál es la finalidad, y por qué aplica en este caso el control en impedancia? Existen varias razones para utilizar este tipo de control, en vez de un control clásico.

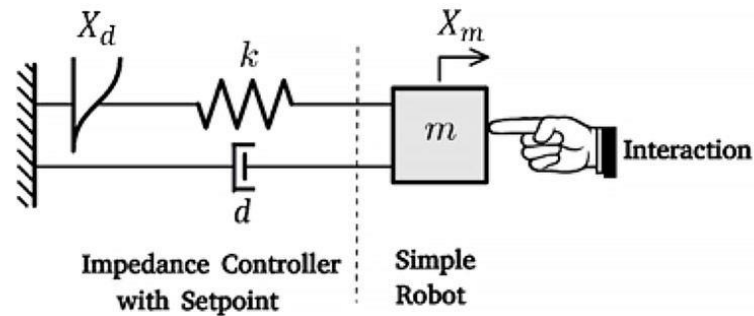


Figura 2.1: Sistema masa-muelle-amortiguador, principio fundamental del control en impedancia

2.1.1. Robótica manipuladora

Según la norma ISO 8373:2012 [2], se define un robot industrial manipulador como: “Un manipulador multipropósito, reprogramable y controlado automáticamente, programable en tres o más ejes, que puede ser fijo o móvil para su uso en aplicaciones de automatización industrial”. Desde el punto de vista del modelado mecánico, Spong et al. [3] definen un manipulador robótico como una cadena cinemática abierta compuesta por una secuencia de cuerpos rígidos (eslabones) conectados mediante articulaciones. Estas articulaciones, típicamente de tipo rotacional o prismático, permiten el movimiento relativo entre eslabones consecutivos, dotando al efector final de la capacidad para posicionarse y orientarse en el espacio de trabajo. Comúnmente este tipo de robot se conoce como brazo robótico, ya que tiene la misma morfología que un brazo humano, pero normalmente con un mayor número de grados de libertad (*dof*).

Las articulaciones, a nivel físico, son actuadores del robot, que permiten realizar un movimiento rotacional o prismático. A nivel rotacional suelen ser esencialmente servomotores, un tipo de actuador que ofrece un movimiento rotacional 1D (también existen articulaciones esféricas, cilíndricas, etc, con mayor *dof*). Lo más común es encontrarse con robots manipuladores de 6 o 7 articulaciones rotacionales, es decir, robots 6R y 7R. Es el caso del robot de la **Figura 2.2**, donde se puede observar la cadena cinemática del mismo. Los ejes representados en la imagen se corresponden a las articulaciones del robot.

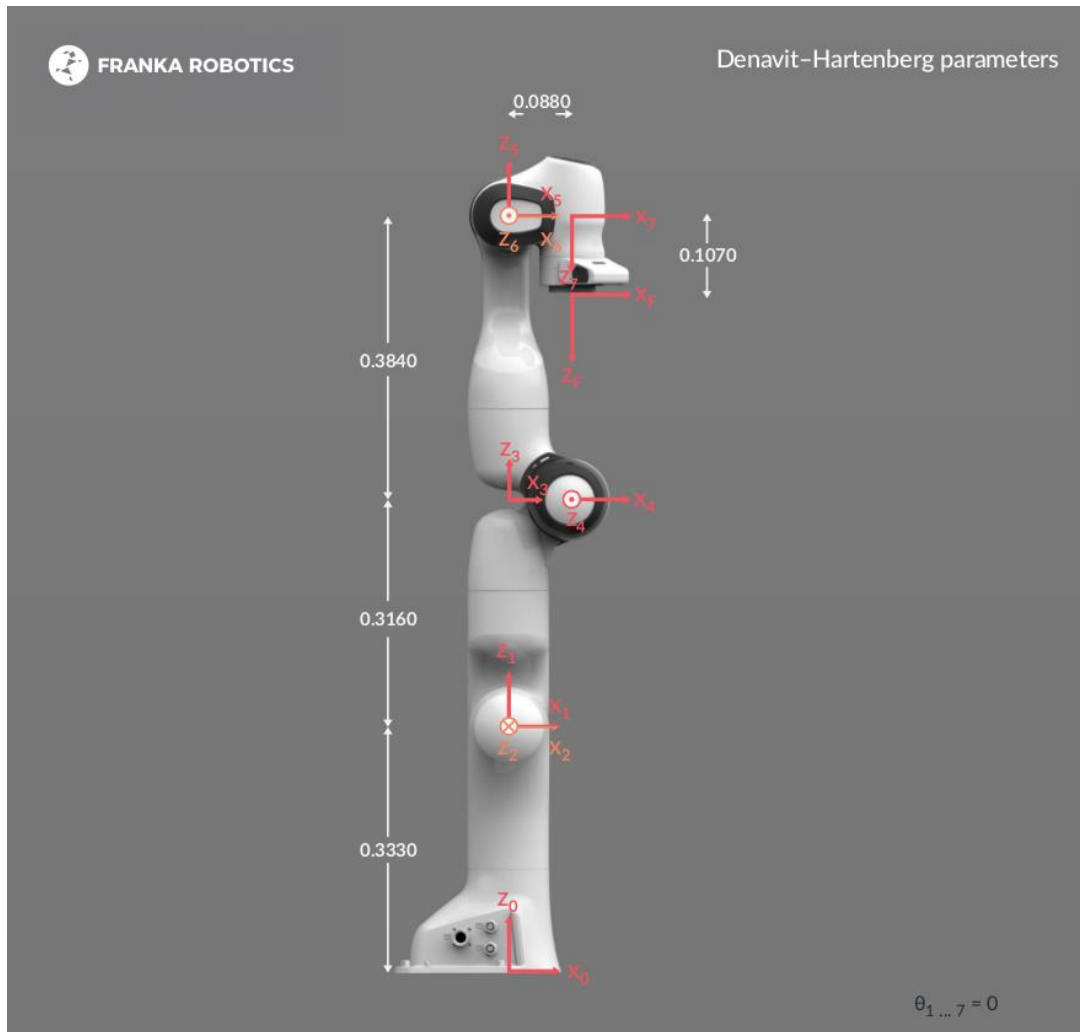


Figura 2.2: Cadena cinemática del robot Franka Research 3, un robot 7R

A nivel de control, las estrategias más habituales son 2: control articular y control cartesiano. En el caso del control articular, le indicamos a cada servomotor del robot la rotación que tiene que realizar para llegar a una pose deseada. Este tipo de control es el más directo para un robot manipulador normal, ya que se mueve en ese espacio de trabajo. Por ende, para tareas repetitivas, donde el robot siempre tiene que situarse en las mismas poses, se suele utilizar este tipo de control, ya que es preciso y directo.

No obstante, este esquema de control no es intuitivo para los humanos, y no aporta información sobre dónde está situado el robot en el entorno. Para tareas no repetitivas, que no tiene que acabar en la misma pose, o que no conoce a priori la pose meta; por ejemplo, *pick & place* donde el objeto se pueda mover, se suele trabajar en el espacio cartesiano. Sabiendo dónde se quiere situar el *end effector* del robot dentro del entorno, se calcula la configuración articular deseada del robot para alcanzar dicha pose, mediante un algoritmo de cinemática inversa.

Por otro lado, como se había introducido en el contexto del proyecto, ahora mismo se pretende realizar una tarea de control visual para situar al robot encima del objeto, pero esa no suele ser la finalidad de un robot manipulador. Al fin y al cabo, su propio nombre define su funcionalidad, la manipulación de uno o varios objetos. Al manipular un objeto, puede resultar crítico conocer la dinámica y las fuerzas que hay que ejercer sobre el objeto o sobre el entorno, por ejemplo, si el robot tiene que realizar una soldadura o asegurar con tornillos una superficie. Es aquí donde entra el control en impedancia, ya que como se ha explicado en el punto anterior, esta estrategia de control nos aporta información sobre la dinámica del robot y la interacción física con el entorno.

En el futuro del proyecto se llevará a cabo una tarea de manipulación donde será necesaria una precisión muy elevada, por lo que tras finalizar este TFG se requerirá de un control en impedancia que funcione adecuadamente, para poder incorporar este tipo de interacción con el entorno al comportamiento.

2.1.2. Robótica colaborativa

Para la implementación efectiva de esquemas de interacción física, es fundamental contar con manipuladores diseñados específicamente para la colaboración humano-robot, los robots colaborativos (o cobots). Este tipo de robots funcionan de distinta manera a los brazos robots convencionales, ya que se conocen los torques generados en las articulaciones del brazo (ya sea por sensores fuerza/par o por estimación a partir de la corriente del motor). Gracias a esto conocemos la dinámica del robot, y como he explicado anteriormente, el control en impedancia utiliza la dinámica y la interacción con el entorno para realizar el control. Por ende, al conocer en todo momento los torques generados por el robot, se puede detectar cualquier manipulación con el entorno y/o colisión con algún elemento o persona. Es por eso por lo que este tipo de robots se han utilizado mucho en industria hoy en día, ya que ofrecen una mayor seguridad en entornos donde haya una interacción con el humano (garantizando la seguridad intrínseca requerida por normativas como la ISO/TS 15066 para espacios de trabajo compartidos [4]), al mismo tiempo que una gran precisión en la manipulación de objetos.

Entonces, dado que un robot colaborativo se controla en impedancia, debido a la tipología del robot, ¿qué hay de novedoso? Es cierto que un robot colaborativo siempre es controlado en impedancia, pero realmente este tipo de control normalmente lo utiliza el propio controlador de bajo nivel del robot, y es el usuario el que crea un controlador cinemático (en espacio articular o cartesiano) el cual está supeditado al software del robot. Entonces, al hacer un control en impedancia directamente, eliminamos este segundo controlador, y trabajamos directamente en el espacio en el que trabaja el sistema robótico, lo que implica una mayor eficiencia y velocidad de computación.

2.2. Definición del control visual

La fundamentación teórica del control visual presentada a continuación, y en general en este Trabajo Fin de Grado, se basa en la información que concierne al capítulo 6 del libro *Robot Manipulators: Modeling, Performance Analysis and Control* [5]. Este capítulo ha sido escrito por François Chaumette, investigador en Inria (*Institut national de recherche en sciences technologies du numérique*), organización responsable de la plataforma de control visual ViSP, utilizada en la parte práctica de este proyecto. Este capítulo contiene una explicación ampliamente detallada sobre la definición del control visual, al igual que una matemática rigurosa de los esquemas de control.

En robótica cada vez es más frecuente el uso de diferentes tipos de sensores exteroceptivos que proporcionen información sobre el entorno donde opera el robot, ya que esta información puede resultar imprescindible para la tarea. Por ejemplo, en robótica móvil, se utilizan diferentes tipos de dispositivos para percepción y localización, como balizas pasivas, marcas viales o GPS; pero sistemas de visión artificial y telemetría destacan por su versatilidad, ya que pueden emplearse en tareas de localización, navegación y exploración.

Durante muchos años, la reconstrucción 3D del entorno se creía totalmente necesaria para cualquier movimiento del robot en entornos parcialmente desconocidos. Sin embargo, gracias a los avances algorítmicos y tecnológicos se ha podido integrar más eficientemente la percepción y la acción, incorporando directamente las medidas visuales en leyes de control de lazo cerrado. Este enfoque es lo que se conoce como control visual o *visual servoing*. Utilizamos la información proporcionada por una o varias cámaras para controlar el movimiento de un sistema robótico. El objetivo es utilizar un conjunto de variables visuales, estableciendo un determinado valor deseado, para definir una ley de control que intente minimizar un error. Esta implementación permite llevar a cabo tareas de posicionamiento, realización de trayectorias, o, lo que nos concierne a nosotros en este trabajo, seguimiento de objetos.

2.2.1. Eye-in-hand vs eye-to-hand

Se puede considerar que hay 2 tipos de entorno en control visual: cuando la cámara está montada *eye-in-hand* o *eye-to-hand* (mostrado en la **Figura 2.3**). El primer caso quiere decir que tenemos una cámara instalada en el *end effector* del robot, por ende, se observa el entorno desde el punto de vista del robot. Esto deriva en una implementación más intuitiva, ya que el movimiento del robot afecta directamente al movimiento de la vista. Además, la precisión es mucho mayor, ya que la visión está condicionada por el acercamiento al objeto. Por

otro lado, con una (o varias) cámaras montadas *eye-to-hand* tenemos un mayor campo de visión del espacio de trabajo, ya que tenemos una vista panorámica. Sin embargo, se pueden presentar problemas de oclusión, de que el propio robot oculte la visión del objeto en movimiento.

En consecuencia, la aplicación de la tarea suele definir la configuración del entorno. En tareas en las que el robot tiene que manipular de cerca el objeto (*grasping*, soldadura, inspección, seguimiento) se suele colocar la cámara en el efector final del robot. En tareas de *pick & place* rápido o evitación de colisiones, se suelen colocar una o varias cámaras fijas, que permiten obtener un mayor campo de visión de la escena.

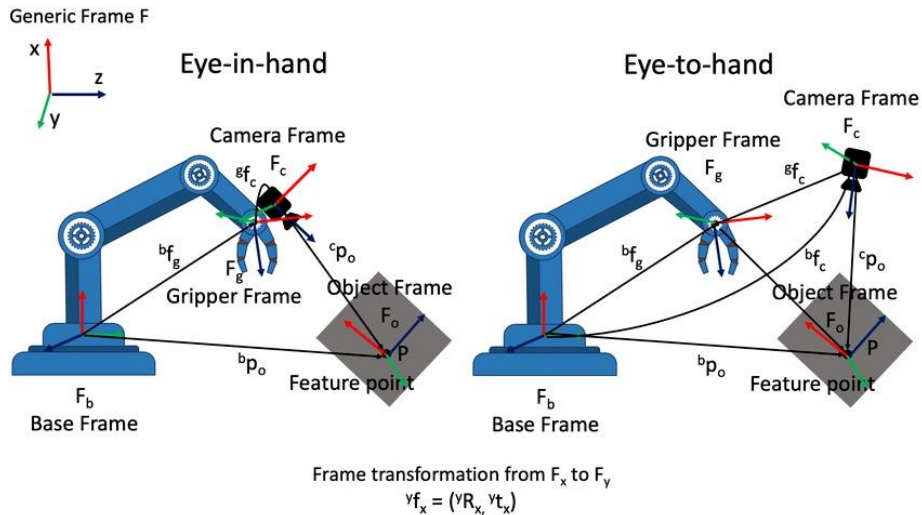


Figura 2.3: Escena con cámara *eye-in-hand* frente a *eye-to-hand*

2.3. Cámaras de eventos

Tradicionalmente, el control visual se ha basado en cámaras convencionales (*frame-based*) que capturan imágenes completas a una frecuencia fija (típicamente 30 o 60 Hz). Este enfoque presenta limitaciones inherentes en aplicaciones de robótica de alta velocidad o en condiciones de iluminación difíciles, debido a problemas de latencia, desenfoque de movimiento (*motion blur*) y rango dinámico limitado.

En contraposición a este paradigma, en los últimos años ha cobrado fuerza el uso de la visión basada en eventos (o visión neuromórfica). Estos sensores, inspirados en la biología de la retina, no capturan frames completos, sino que transmiten información de forma asíncrona solo cuando detectan un cambio de intensidad en un píxel (ver reconstrucción de imagen mediante eventos en la **Figura 2.4**).

Como se detalla en el exhaustivo análisis de Gallego et al. [6], este principio de funcionamiento otorga a las cámaras de eventos ventajas significativas frente a las cámaras estándar:

1. **Resolución temporal en el orden de microsegundos:** Permitiendo un control de muy baja latencia.
2. **Alto Rango Dinámico (HDR):** Capacidad para operar en escenas con cambios drásticos de iluminación (superando los 120 dB).
3. **Bajo consumo de energía y ancho de banda:** Al transmitir solo los cambios de la escena, se evita el procesamiento de información redundante.

Estas características posicionan a la visión basada en eventos como una solución prometedora para superar las limitaciones de estabilidad y convergencia en esquemas de control visual de alta dinámica.



Figura 2.4: Diferencia entre imagen con una cámara convencional y una cámara de eventos. **Fuente:** WEB de Prophesee [7]

2.4. Aprendizaje por refuerzo

El Aprendizaje por Refuerzo (RL) es un área del aprendizaje automático centrada en cómo los agentes deben tomar acciones en un entorno para maximizar alguna noción de recompensa acumulada. Según definen Sutton y Barto [8], el RL se diferencia del aprendizaje supervisado en que no requiere pares de entrada-salida etiquetados explícitamente; en su lugar, el agente aprende mediante la interacción directa con el entorno, basándose en una señal de recompensa escalar que evalúa la calidad de las acciones tomadas.

No obstante, como señalan Kober et al. [9], la aplicación de estas técnicas en robótica presenta desafíos únicos debido a la alta dimensionalidad de los espacios de estados y acciones (grados de libertad del manipulador), así como la necesidad de garantizar la seguridad del hardware durante la fase de exploración.

En este TFG se presenta una tarea de Aprendizaje por Refuerzo Profundo (Deep RL), ya que, dada la complejidad de la tarea y el entorno, es necesario acudir a una implementación más resolutive.

2.4.1. Aprendizaje por refuerzo profundo (Deep RL)

Para entender la implementación posterior de la tarea de Deep RL es necesario proporcionar una explicación detallada del funcionamiento de este tipo de algoritmos, en qué se diferencian con los algoritmos clásico de RL y cómo funcionan. Para ello, se ha utilizado el material de la asignatura de robótica adaptativa del grado [10]. Esta explicación se puede desglosar en 4 puntos fundamentales del Aprendizaje por refuerzo profundo:

2.4.1.1. Proceso de decisión de Markov (MDP)

El marco matemático del aprendizaje por refuerzo es el proceso de decisión de Markov, que define cómo se modela la interacción entre el agente (el robot) y el entorno. Para esto, se asume que se verifica la propiedad de Markov, lo que quiere decir que la probabilidad de que el sistema esté en el estado s_{t+1} , solo depende del estado actual s_t y la acción ejecutada a_t , y no del historial de interacciones entre el entorno y el agente. Entonces, tenemos que un MDP se define como una tupla $M = (S, A, P, R, \gamma)$.

- **S (Espacio de estados):** Conjunto de todas las configuraciones posibles del entorno.
- **A (Espacio de acciones):** Conjunto de decisiones posibles.
- **P (Dinámica):** La probabilidad de transición $P(s_{t+1}|s_t, a_t)$ que describe cómo evoluciona la física del sistema.
- **R (Función de recompensa):** Escalar $r_t = R(s_t, a_t)$ que proporciona retroalimentación inmediata sobre la calidad de la acción tomada.
- **γ (Factor de descuento):** Valor $\gamma \in [0,1)$ que determina la importancia de las recompensas futuras

frente a las inmediatas.

2.4.1.2. El objetivo final: la política óptima

El fin del agente es encontrar el comportamiento que maximice la suma de las recompensas a lo largo del tiempo, es decir, la política óptima (π^*). Matemáticamente, el objetivo es encontrar los parámetros θ^* que maximicen el retorno esperado:

$$J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r(s_t, a_t) \right]$$

2.4.1.3. La ecuación de Bellman

Para poder lidiar con el problema de maximización del retorno se utiliza la función de valor de acción (Q-valor), denotada como $Q^\pi(s, a)$. Esta función estima el retorno esperado si a partir del estado s , se toma la acción a , y a partir de entonces se sigue la política π . La propiedad fundamental que permite el aprendizaje iterativo es que esta función satisface una estructura recursiva conocida como la ecuación de Bellman:

$$Q^\pi(s_t, a_t) = \mathbb{E}_{s_{t+1} \sim P} [r(s_t, a_t) + \gamma Q^\pi(s_{t+1}, \pi(s_{t+1}))]$$

Esta ecuación varía dependiendo de si la política seguida π es determinista o estocástica, y si el entorno es determinista o estocástico.

2.4.1.4. Redes neuronales

En problemas clásicos con espacios de estados pequeños y discretos, sería posible almacenar el valor de cada acción en una tabla (*Tabular RL*). Sin embargo, en robótica el espacio de estados es continuo y de alta dimensión (infinitas posiciones poses del brazo), lo que hace inviable el uso de tablas ("La maldición de la dimensionalidad").

Aquí es donde entra el Deep Learning. Las redes neuronales actúan como aproximadores universales de funciones. En lugar de memorizar, la red neuronal parametriza por un vector de pesos θ aprende una función $\pi_\theta(s)$ capaz de generalizar, es decir, puede predecir la acción óptima incluso para estados s que nunca ha visto exactamente antes, basándose en patrones aprendidos de la geometría y dinámica del robot.

Para que la red neuronal aprenda es necesario ajustar sus pesos θ iterativamente para reducir el error de sus predicciones. Este proceso se realiza a partir del algoritmo del Descenso del Gradiente. El proceso consiste en calcular la derivada de una función de pérdida (*loss function*) con respecto a los parámetros de la red. Utilizando el algoritmo de *backpropagation* se actualizan los pesos de la red neuronal en la dirección opuesta al gradiente, con el fin de minimizar el error. A veces, esta *loss function* no define cómo de mal ha estimado la red, sino que se calcula a partir de la recompensa positiva. Por ende, cuando esto ocurre, necesitamos ir en la dirección positiva del gradiente, y tenemos que cambiar el símbolo a esta función.

Algunos de los algoritmos más conocidos y utilizados de Deep RL son DQN [11], A2C [12] o DDPG [13].

3. ESTADO DEL ARTE

Para comprender y justificar la innovación que se presenta en este TFG es necesario considerar primero las investigaciones publicadas actualmente y definir cuál es el estado del arte actual sobre el control en impedancia y el control visual.

3.1. Control en impedancia

El control en impedancia es el campo donde transcurre la mayor innovación que presenta este Trabajo Fin de Grado, y a lo largo de este apartado se explicará el problema actual que presenta dicha estrategia, cómo se ha abordado hasta ahora y cuál es el planteamiento que se presenta.

3.1.1. Problema actual del control en impedancia

En lo relativo al propio control en impedancia no se presentan novedades frente a lo establecido por Hogan en 1985 [1]. La razón de esto es física, la ley de control de Hogan intenta imponer una dinámica de masa-muelle-amortiguador como la mostrada en la **Figura 2.1**. Como la física newtoniana no cambia, esta ecuación sigue siendo la forma más efectiva y natural de modelar una interacción estable. A día de hoy no se ha encontrado una mejor analogía física que el sistema que presenta Hogan.

Por otro lado, en la propia implementación del control en impedancia es donde se producen las innovaciones. Para ello, vamos a observar la ley de control básica sobre la que se fundamenta el control en impedancia, el sistema masa-muelle-amortiguador:

$$F_{ext} = M(\ddot{x} - \ddot{x}_d) + D(\dot{x} - \dot{x}_d) + K(x - x_d)$$

Esta ley será explicada en más profundidad en el apartado de desarrollo del trabajo, pero ahora mismo sirve para poner en contexto las ganancias del sistema, K y D . Estas ganancias modifican el comportamiento del sistema, siendo K la encargada de modificar el comportamiento de muelle, y D el comportamiento de resorte. En control en impedancia se conocen como rigidez (*stiffness*) y amortiguamiento (*damping*).

El control en impedancia clásico utiliza ganancias constantes, pero este no es un enfoque adecuado. Dentro de la misma tarea se pueden necesitar diferentes comportamientos del robot, incluso adaptando individualmente las ganancias a cada articulación. En líneas generales, si se pone el robot muy rígido, es preciso pero peligroso al entrar en contacto con el entorno, pero si se pone muy blando es impreciso, aunque más seguro. Además, para determinadas tareas en las que tienes que ejercer una determinada fuerza, y de una determinada manera, sobre un objeto o superficie, esta implementación no nos sirve.

3.1.2. Control de impedancia variable

La evolución más directa de la teoría clásica, y la primera innovación, es el control de impedancia variable (VIC). En esta implementación, se intenta copiar el comportamiento del ser humano. Para un movimiento lineal y de gran precisión, como enhebrar una aguja, el humano tensa el brazo (alta impedancia); pero cuando se quiere hacer un movimiento más ligero, como saludar con la mano, entonces se relaja (baja impedancia). En el artículo de Albu-Schäffer et al. [13] se explica cómo los parámetros dejan de ser constantes y pasan a ser funciones del tiempo (planificación) o del estado (reflejos): $K(t)$ o $K(x)$.

En el artículo mencionado [13] se presenta una arquitectura de control pasiva. A grandes rasgos, se presenta cómo diseñar D cuando se cambia en tiempo real K sin que el robot se vuelva inestable. Se propone un diseño de amortiguamiento por factorización, que se basa en términos de teoría de control para, según la variabilidad de K , poder adaptar D de forma segura. También habla de la condición de pasividad para saber cómo variar la rigidez del sistema adecuadamente. Por otro lado, al utilizar robots redundantes (7 *dof*) se define una segunda

rigidez dependiente del estado, la rigidez del espacio nulo. Esta rigidez no afecta a la mano, sino al codo. Permite que se empuje el codo del robot y este se comporte como un muelle, sin alterar la pose de la mano.

El problema presente en esta implementación recae en la propia metodología utilizada. Lo que se aporta son unas pautas para definir los parámetros necesarios de manera segura, y ayudar al técnico responsable de la elección de los mismos, pero no describe las pautas para elegir los parámetros más eficientes para una tarea en concreto, un algoritmo que según la tarea y el estado del robot se adapte y devuelva en cada momento los valores más adecuados.

3.1.3. Control de impedancia adaptativo

El control de impedancia variable es una estrategia de control proactiva. Es el ingeniero quien define cómo quiere que cambien las variables del sistema, mediante una estrategia interna. En contraposición a esta estrategia, está el control de impedancia adaptativo (AIC), el cuál es una estrategia reactiva. En este caso, la fuente del cambio no es una estrategia interna del usuario o una planificación, sino que proviene de un error.

El Control de Impedancia utiliza algoritmos de estimación en tiempo real basados en modelos dinámicos. Un enfoque clásico [14] [15] consiste en utilizar estimadores como Mínimos Cuadrados Recursivos (RLS) para identificar la rigidez del entorno (K_e) y adaptar las ganancias del controlador (K_d y D_d) mediante leyes de actualización algebraicas que garantizan la estabilidad según el criterio de Lyapunov.

El problema de esta implementación es su independencia a la tarea. La rigidez y el amortiguamiento varían para ofrecer estabilidad al robot, pero no se adaptan a la tarea para ofrecer la mayor eficiencia en términos de fuerza y precisión.

3.1.4. Control de impedancia variable mediante aprendizaje

Para solucionar los problemas de las implementaciones anteriores se ha desarrollado una estrategia de control de impedancia variable, pero mediante aprendizaje por refuerzo (*Learning Variable Impedance Control*). En vez de utilizar fórmulas cerradas de control clásico, se implementa una tarea de RL, que permite al sistema explorar y adaptarse a una tarea de manera específica. De esta manera, generando una función recompensa (*reward function*) adecuada para la tarea, el robot es capaz de aprender a variar K y D de la manera más precisa posible.

En trabajos como los de Jonas Buchli et al. [16] se aporta un enfoque diferente al utilizado en robótica industrial. Hasta ese momento, se pretendía que el robot fuese rígido siempre, y blando solo en caso de colisión, pero en este artículo se pretende que el robot sea lo más blando posible, con la finalidad de gastar poca energía y ser seguro. Tendrá un comportamiento rígido cuando la tarea lo exija estrictamente.

Por otro lado, la mayor aportación de este artículo reside en utilizar un algoritmo de aprendizaje por refuerzo para la obtención de los parámetros. En este caso, se propone PI^2 (*Policy Improvement with Path Integrals*), un algoritmo estocástico que no utiliza redes neuronales. El robot intenta la tarea varias veces con variaciones aleatorias en su rigidez. Evalúa una función de coste (*cost function*), y ajusta su perfil de impedancia para la siguiente prueba. La función de coste aporta información al algoritmo de cómo de bien ha sido la implementación, intentando reducir dicha función al mínimo. En este artículo esta función de coste está compuesta por el error de la trayectoria y el coste de rigidez (gasto energético).

Existen otras implementaciones mucho más recientes [17] que sí utilizan algoritmos de Deep RL, es decir, con redes neuronales. En esta tesis se utiliza el algoritmo TD3 [18] con el objetivo de adaptar los parámetros para una tarea de *pick & place* y manipulación de objetos. Se propone hacer una separación de la tarea para abordarla en 2 fases: aproximación al objeto y manipulación. Por ende, utilizando el mismo algoritmo, se diseñan 2 aprendizajes diferentes, el primero pensado para aproximar el *end effector* del robot al objeto, y el segundo para coger el objeto y manipularlo. En la primera fase, que es la que nos concierne en este TFG, se utiliza la siguiente *reward function*:

$$r_s = \min_{K_d(t)} \sum_{t=1}^T -100 \times \|\mathbf{x}_d - {}^m \mathbf{x}(t)\|$$

Esta recompensa intenta minimizar la rigidez del sistema al mismo tiempo que el error en la pose del robot.

3.1.5. Propuesta de innovación en el control de impedancia variable mediante aprendizaje

Para el control en impedancia es esencial establecer unas ganancias acordes a la tarea y al movimiento que se ejerce, por ende, la implementación óptima resulta ser la implementación de una tarea de RL para que la estrategia de control adapte dinámicamente estos parámetros, para ofrecer la mayor eficiencia en el control. En este TFG, se quiere explorar esta vía, implementando un algoritmo de Deep RL para encontrar estos parámetros para la tarea propuesta, seguimiento de un objeto.

Por otro lado, se han encontrado aspectos a mejorar en lo relativo a la tesis anteriormente explicada [18], empezando por que esta implementación se centra sobre todo en la obtención de los parámetros para la manipulación del objeto. En cambio, este trabajo se centra exclusivamente en la realización de la trayectoria del robot y el seguimiento del objeto. Además, el aspecto más crítico de esta implementación es que simplemente se intenta minimizar el error de la trayectoria, pero no se tiene en cuenta ningún aspecto dinámico de la misma. En este TFG se propone un nuevo enfoque del comportamiento del agente, modificando así la función recompensa del aprendizaje, para añadir la dinámica del robot. De esta forma, no solo tenemos en cuenta que el robot llegue a la pose meta, sino que también le damos importancia a cómo ha sido este movimiento, minimizando oscilaciones, pero también el tiempo que tarda en llegar a la meta.

3.2. Control visual

El control visual es un campo que ha estado en constante innovación en los últimos años. En este apartado se tratará de explicar la historia evolutiva del control visual, explicando cada enfoque diferente, y cuál es el que se utilizará en este TFG. La fuente de la información de este apartado proviene del primer artículo, con explicaciones más cualitativas, de François Chaumette [20].

3.2.1. Control visual clásico: IBVS y PBVS

El enfoque clásico del control visual se divide en 2 filosofías opuestas según dónde se calcula el error: PBVS (*Position-Based Visual Servoing*) y IBVS (*Image-Based Visual Servoing*).

- **PBVS:** La información visual se utiliza para estimar la pose tridimensional del objeto con respecto a un sistema de coordenadas de referencia (normalmente la base del robot), y el error se calcula y minimiza en el espacio cartesiano. El procedimiento se basa en extraer las características de la imagen, y mediante un modelo geométrico, se localiza matemáticamente el objeto dentro del entorno. Entonces, se utiliza esta pose para calcular el error, que será lo que mueva al robot.
- **IBVS:** En este caso se omite la reconstrucción 3D. El control se realiza directamente en el plano imagen. El objetivo es mover el robot para que las características visuales actuales coincidan con las deseadas en la imagen. En esta implementación no se sabe dónde está el objetivo en metros, simplemente se hace coincidir con cómo se debería de ver el objetivo cuando el robot llega a la meta. Es aquí donde se implementa la matriz de interacción $\mathbf{L}_s$.

En la **Tabla 3.1** se presenta una comparativa de las principales características de las 2 implementaciones de control visual clásico.

Tabla 3.1: Comparación entre PBVS y IBVS

Característica	PBVS (<i>Position-Based</i>)	IBVS (<i>Image-Based</i>)
Espacio de error	Cartesiano 3D	Imagen 2D
Requisito de modelo	Alto (Modelo CAD + calibración precisa)	Bajo (Modelo aproximado / Profundidad estimada)
Trayectoria en 3D	Línea recta (Óptima)	Curva / Impredecible
Estabilidad	Sensible a errores de calibración	Robusto a errores de calibración
Problema principal	Sensibilidad a parámetros	Singularidades de la matriz L_s

3.2.2. Control visual híbrido (2.5D Visual Servoing)

Como se ha podido observar en el apartado anterior, tanto el control visual basado en imagen como en posición presentan diferentes problemas. Ahora el objetivo ya no es solo moverse, sino hacerlo con estabilidad y visibilidad. El IBVS clásico ejecuta trayectorias caóticas, realizando movimientos bruscos y largos para corregir pequeños errores en la imagen. Por otro lado, PBVS es muy sensible a la calibración, lo que provoca error de posicionamiento final. Además, necesita que se le aporte mayor información sobre el objeto a seguir, para poder estimar correctamente su posición.

Para mitigar estas limitaciones en la estrategia se propuso un nuevo enfoque [21], quedarse con las mejores características de ambas implementaciones, desarrollando así el concepto de *2.5D Visual Servoing*. Este término “2.5D” se refiere a que no se utiliza únicamente imagen (2D) ni pose (3D), sino una mezcla desacoplada de ambas. Por una parte, se controla la rotación basándose en una estimación parcial 3D, y por otra, la traslación mediante coordenadas 2D en la imagen. El vector de rotación se controla en el espacio cartesiano para asegurar que el robot gire por el camino más corto, mientras que el centro del objeto se mantiene en la imagen para asegurar una aproximación precisa. Gracias a esto, y tal como se puede observar en la **Figura 3.1**, se obtienen unos mejores resultados que utilizando IBVS o PBVS clásico.

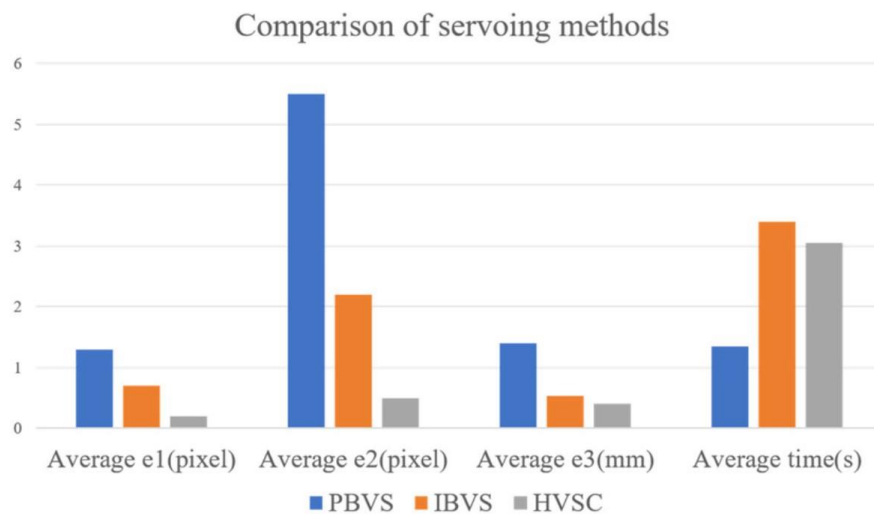


Figura 3.1: Resultados de PBVS, IBVS y HVSC (híbrido). **Fuente:** Li et al., página 15 [22]

3.2.3. Control visual fotométrico

El problema de los métodos anteriores es que dependen de un paso previo crítico: el procesamiento de la imagen. Como se requiere de la extracción de características, el éxito de la tarea depende de la precisión de los diferentes detectores (Canny, Harris, SIFT) para encontrar puntos o bordes. Por ende, en entornos con falta de iluminancia, desenfoque por movimiento rápido o textura muy repetitiva, el detector puede fallar, lo que provoca que el control colapse.

La solución presentada por Collewet y Marchand [23] se trata de utilizar la iluminancia pura de cada píxel como característica visual. El fundamento matemático de este enfoque es la ecuación de flujo óptico, minimizando la diferencia de intensidad píxel a píxel entre la imagen actual y la deseada.

3.2.4. Deep visual servoing

Aunque el control visual fotométrico elimina la necesidad de extraer características geométricas manualmente, sigue dependiendo de modelos de interacción luz-movimiento (ecuación del flujo óptico), por ende, es sensible a cambios bruscos de iluminación externa (sombras que aparecen de repente). Eso nos lleva a la siguiente implementación: *Deep visual servoing*.

Las nuevas líneas de investigación [24] buscan sustituir los modelos matemáticos anteriores por redes neuronales convolucionales (CNN). El objetivo es aprender la relación entre la imagen y el movimiento mediante entrenamiento con datos masivos, permitiendo operar en entornos complejos, donde se dificulta la extracción de características.

A pesar del potencial de estos métodos basados en aprendizaje, carecen a menudo de las garantías de estabilidad y precisión geométrica que ofrecen los métodos analíticos anteriores.

3.2.5. Control visual dinámico

El control visual dinámico [25] es la estrategia de control que considera las fuerzas, pares e inercia del robot dentro del bucle de visión, en lugar de asumir una respuesta cinemática perfecta. Mientras que el objetivo del resto de enfoques de control visual es devolver una velocidad deseada para el robot, el control visual dinámico devuelve los torques deseados que reducen el error visual.

Los principales problemas que intenta solucionar este tipo de planteamiento es la corrección del movimiento a altas velocidades, ya que si el robot se mueve muy rápido es imperativo considerar la inercia) y la dinámica no lineal en robots pesados. Para solucionar esto existen 2 arquitecturas principales:

- **Estructura en cascada:** Por un lado, hay un bucle externo de visión que calcula velocidades, y por otro un bucle interno de dinámica que se asegura de que el robot cumpla esas velocidades.
- **Control dinámico directo:** El error visual entra directamente en la ecuación dinámica del robot, sin pasar por un controlador de velocidad intermedio.

4. HERRAMIENTAS Y RECURSOS

A continuación, se presentarán las herramientas y recursos que se utilizarán a lo largo del desarrollo de este Trabajo Fin de Grado. Se dividirá en hardware y software, explicando de qué se trata cada uno de los elementos y el por qué de su elección.

4.1. Hardware

Las restricciones impuestas por el hardware utilizado en el proyecto serán importantes para definir la metodología que se seguirá a lo largo del trabajo. A continuación, se explicará la infraestructura experimental con la que se cuenta. En este apartado no se justificará la elección de las herramientas, ya que en este caso son de las que se dispone y definen el objetivo de este proyecto.

4.1.1. Robot

El robot manipulador que se va a utilizar en este proyecto es el Franka Research 3 (FR3) [26]. Este robot, mostrado en la **Figura 4.1**, es desarrollado por Franka Robotics, y se trata de un robot colaborativo diseñado específicamente para tareas de investigación y manipulación sensible. Es un robot 7R (7 articulaciones rotacionales), por lo que tiene 7 grados de libertad. Esta redundancia permite el movimiento en el espacio nulo, es decir, reorientar el codo del robot para evitar obstáculos o colisiones sin alterar la pose del efector final. Por otro lado, dispone de sensores de par en cada una de las articulaciones. De este modo, el control del robot se vuelve mucho más preciso que simplemente calculando una estimación mediante la corriente suministrada por los motores (explicado en el Apartado 2.1.2).

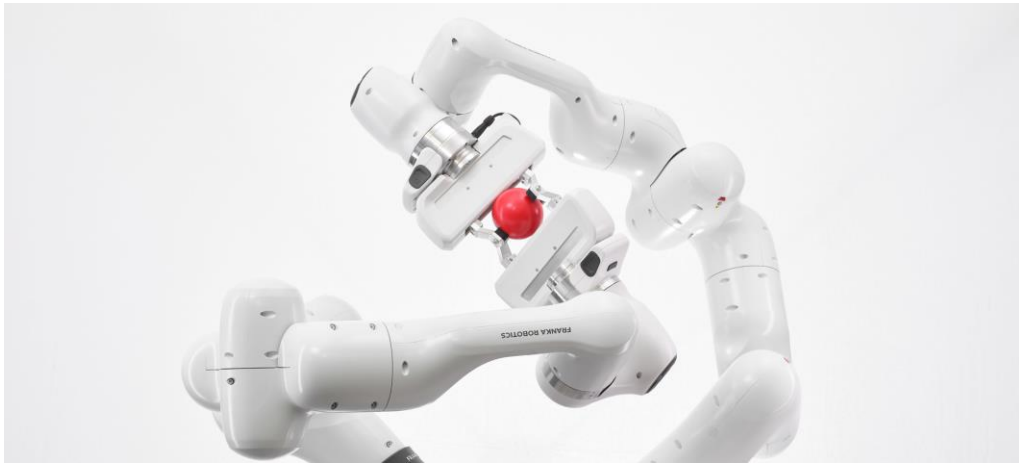


Figura 4.1: Robot colaborativo Franka Research 3

Otra característica del robot realmente relevante para este proyecto es la arquitectura de control. Utiliza *Franka Control Interface* (FCI), una interfaz que permite una comunicación bidireccional de baja latencia, a una frecuencia de 1kHz. Esta alta frecuencia es vital para explotar la baja latencia de la cámara de eventos, al igual que para el bucle de control dinámico y de visión, ya que permite enviar comandos de par, velocidad o posición directamente a los controladores de bajo nivel.

Finalmente, otra propiedad que resulta de especial interés es la velocidad y los límites, tanto en el espacio cartesiano como en el articular, del robot. Estas especificaciones vienen reflejadas en la **Tabla 4.1** y la **Tabla 4.2**.

Tabla 4.1: Límites del espacio cartesiano del FR3 [27]

Magnitud	Traslación	Rotación	Codo
$\dot{p}_{max}$	$3.0 \frac{m}{s}$	$2.5 \frac{rad}{s}$	$2.620 \frac{rad}{s}$
$\ddot{p}_{max}$	$9.0 \frac{m}{s^2}$	$17.0 \frac{rad}{s^2}$	$10.0 \frac{rad}{s^2}$
$\dddot{p}_{max}$	$4500.0 \frac{m}{s^3}$	$8500.0 \frac{rad}{s^3}$	$5000.0 \frac{rad}{s^3}$

Tabla 4.2: Límites del espacio articular del FR3 [27]

Magnitud	Joint 1	Joint 2	Joint 3	Joint 4	Joint 5	Joint 6	Joint 7	Unidad
q_{max}	2.90	1.84	2.90	-0.12	2.88	4.6216	3.05	rad
q_{min}	-2.90	-1.84	-2.90	-3.08	-2.88	0.4398	-3.05	rad
$\dot{q}_{max}$	2.62	2.62	2.62	2.62	5.26	4.18	5.26	$\frac{rad}{s}$
$\ddot{q}_{max}$	10	10	10	10	10	10	10	$\frac{rad}{s^2}$
$\ddot{q}_{max}$	5000	5000	5000	5000	5000	5000	5000	$\frac{rad}{s^3}$
$\tau_{j_{max}}$	87	87	87	87	12	12	12	Nm
$\dot{\tau}_{j_{max}}$	1000	1000	1000	1000	1000	1000	1000	$\frac{Nm}{s}$
$\dot{q}_{offset}$	0.6599	0.2517	0.2000	0.3533	0.5757	0.4878	0.4628	$\frac{rad}{s}$
$\ddot{q}_{dec}$	6.0	2.585	3.5	4.0	17.0	5.5	17.0	$\frac{rad}{s^2}$

4.1.2. Cámaras

Para este proyecto se utilizarán 2 tipos de cámaras distintas, una *frame-based* y otra *evento-based*. La primera de ellas se trata de la Intel RealSense D405 [28] (ver **Figura 4.2:** Cámara RGBD Intel RealSense D405), una cámara RGBD para corto alcance y alta precisión. Este tipo de cámara ofrece una lectura de la profundidad bastante precisa de $\pm 2\%$ de error a 50cm de distancia, operando idealmente entre 7cm y 50cm de distancia. Pero lo que concierne a este proyecto no es la profundidad, sino el sensor RGB, que ofrece una resolución de 1280 x 720 a 30 fps. En este Trabajo Fin de Grado no vamos a utilizar la componente de la profundidad que ofrece esta cámara, ya que se utilizará para realizar las pruebas necesarias antes de utilizar la cámara de eventos, que es el sensor que define este proyecto.



Figura 4.2: Cámara RGBD Intel RealSense D405

Asimismo, tenemos por otro lado la cámara de eventos, la Prophesee EVK4 [29], presente en la **Figura 4.3** **Figura 4.3:** Cámara de eventos Prophesee EVK4. Esta cámara de eventos dispone de una resolución de 1280 x 720, con una latencia de 220 μ s (≈ 4.5 kHz). Al tener una latencia tan baja, una de las ventajas de esta cámara a la hora del control visual es la supresión del *motion blur*. Además, tiene un rango dinámico superior a 120dB, por lo que permite operar en condiciones de iluminación extremas.

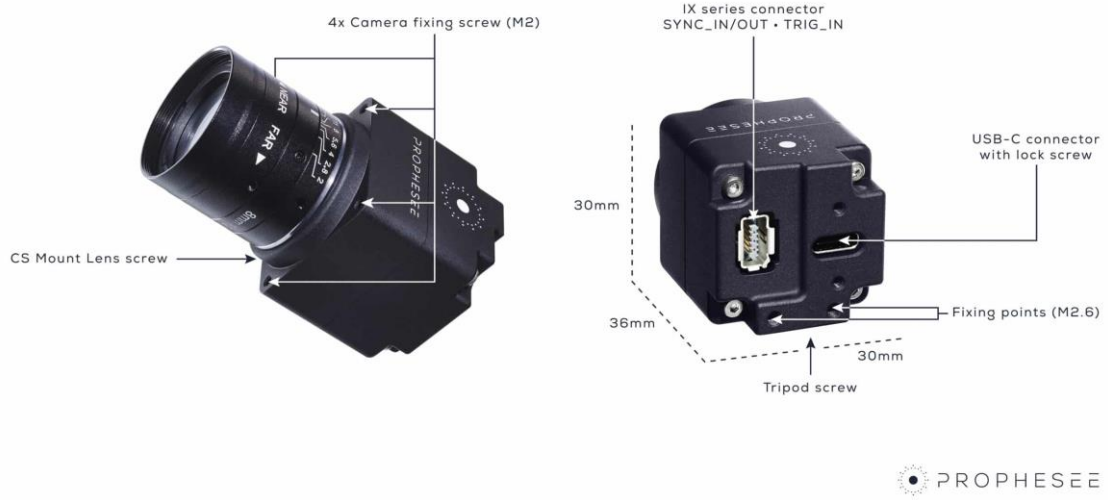


Figura 4.3: Cámara de eventos Prophesee EVK4

4.1.3. Ordenador

A la hora de analizar los resultados y realizar las métricas del trabajo es importante tener en cuenta los recursos computacionales de los que se dispone. Por ende, en la **Tabla 4.3** se recogen las especificaciones del ordenador donde se ha llevado a cabo el proyecto, encargado de realizar todo el procesamiento de las diferentes tareas que se han realizado en este proyecto.

Tabla 4.3: Especificaciones del sistema

Componente	Especificación
CPU	Intel Core i9-10900
GPU	Nvidia GeForce RTX 3070 8GB
Memoria RAM	128 GB
Sistema Operativo	Ubuntu 22.04 LTS

4.1.4. Entorno de trabajo

El entorno de trabajo se compone de los elementos definidos en los apartados anteriores, situados encima de una mesa de trabajo de color negro, para facilitar la detección de los objetos. El objeto que se ha utilizado como ejemplo para la detección y seguimiento ha sido una pelota azul de un diámetro de 6cm. Se ha elegido este objeto ya que permite moverlo con cierta velocidad fácilmente y es invariante a la rotación, ya que al tratarse de una esfera se visualiza igual independientemente del punto de vista o la rotación de la misma. En la **Figura 4.4** y en la **Figura 4.5** se puede observar el entorno de trabajo, con y sin objetos utilizados para la depuración de la detección. **Figura 4.4**

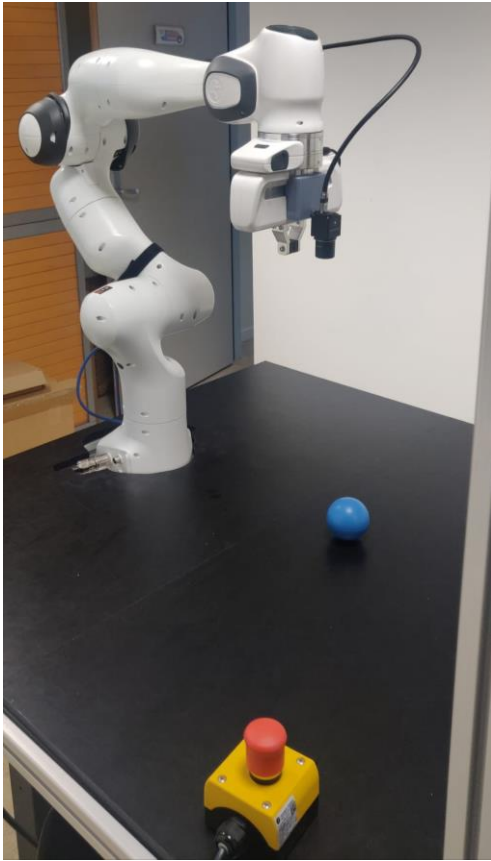


Figura 4.4: Entorno de trabajo sin objetos

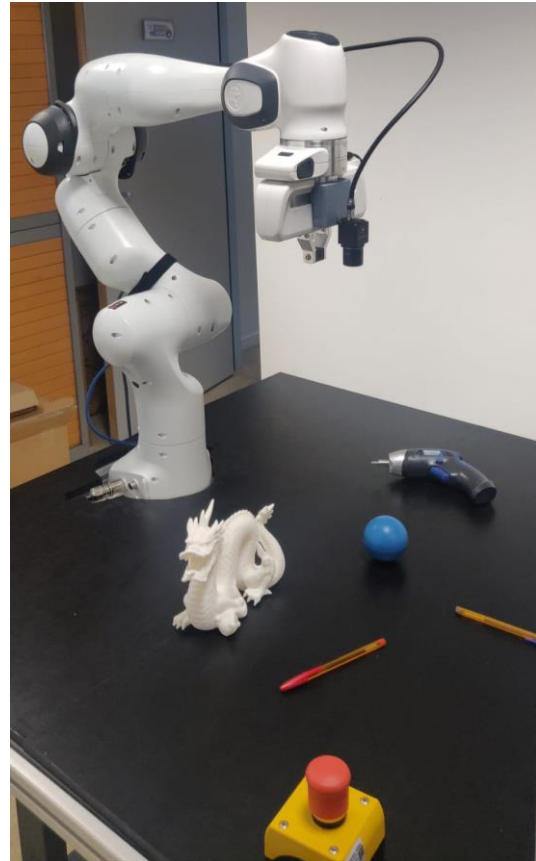


Figura 4.5: Entorno de trabajo con objetos

4.2. Software

En esta sección se detalla todo el software empleado para el desarrollo de proyecto. La elaboración del trabajo depende de varias herramientas software para el control del robot, la simulación, algoritmos de aprendizaje por refuerzo y control visual.

4.2.1. Control del robot: libfranka

La elección de las librerías utilizadas para el control del robot no admite cuestionamiento, ya que se utiliza la propia librería de Franka: *libfranka* [30]. Se ha elegido esta opción ya que permite realizar un control a más bajo nivel en comparación con otras librerías, como *ViSP*, que, aunque puedan facilitar la comprensión del código al operar a más alto nivel, y por debajo utilizar *libfranka*, resulta imperativo asegurar la alta frecuencia de trabajo de 1kHz. Además, al utilizar esta librería se puede llevar a cabo el control en impedancia directamente, utilizando el controlador que utilizará en última instancia el robot.

4.2.2. Simulador: Isaac Sim

En los proyectos de robótica resulta crítico el uso de simuladores, ya que en la fase de depuración del proyecto hay que tener en cuenta el coste e incluso peligro que supone manejar este tipo de hardware. Por ende, la manera más eficaz es empezar la implementación utilizando un simulador, en el cuál se puede manejar el robot libremente y con la confianza de que no se está utilizando material físico.

Hay una amplia diversidad de simuladores para robótica, cada uno con características diferentes. En este proyecto se ha elegido el simulador Isaac Sim de Nvidia [31], debido a varios motivos:

- **Motor de físicas avanzado:** Integra la tecnología *Nvidia Physx 5*, un motor de física avanzado que soporta simulación de cuerpos rígidos, cuerpos deformables y fluidos. En este Trabajo Fin de Grado se pretende llevar a cabo una tarea de control en impedancia, por lo que la precisión en el cálculo dinámico del robot y del entorno resulta un aspecto crítico en la elección del simulador, e Isaac Sim cumple con este requisito. Además, este motor de física permite la simulación masiva en paralelo en GPU, arquitectura propuesta por Makoviychuk et al. [32], por lo que se elimina el cuello de botella de transferencia de memoria con la CPU, reduciendo mucho el tiempo de entrenamiento.
- **Conectividad con ROS 2:** Dispone de una conectividad nativa altamente optimizada con ROS 2, recurso totalmente necesario para la tarea de RL.
- **Renderizado fotorrealista:** Esta es una de las características estrella de este simulador. Aporta una calidad visual de los modelos y del entorno muy avanzada. Además, utiliza tecnología *Ray Tracing* en tiempo real para simular el comportamiento físico de la luz. De este modo, se simulan sombras, reflejos, texturas y condiciones lumínicas complejas con un alto grado de realismo. Esto puede resultar útil en la extracción de características del control visual, ya que además reducirá la brecha *sim-to-real*. La calidad de la imagen se puede apreciar en la **Figura 4.6**.
- **Conectividad con código de control:** Isaac Sim proporciona su propio *workspace* para poder desarrollar los códigos de simulación. Ofrece un alto grado de configurabilidad del entorno a partir desde el propio código, donde se controlan también los diferentes robots y/o elementos del entorno. Además, el lenguaje de programación es Python.

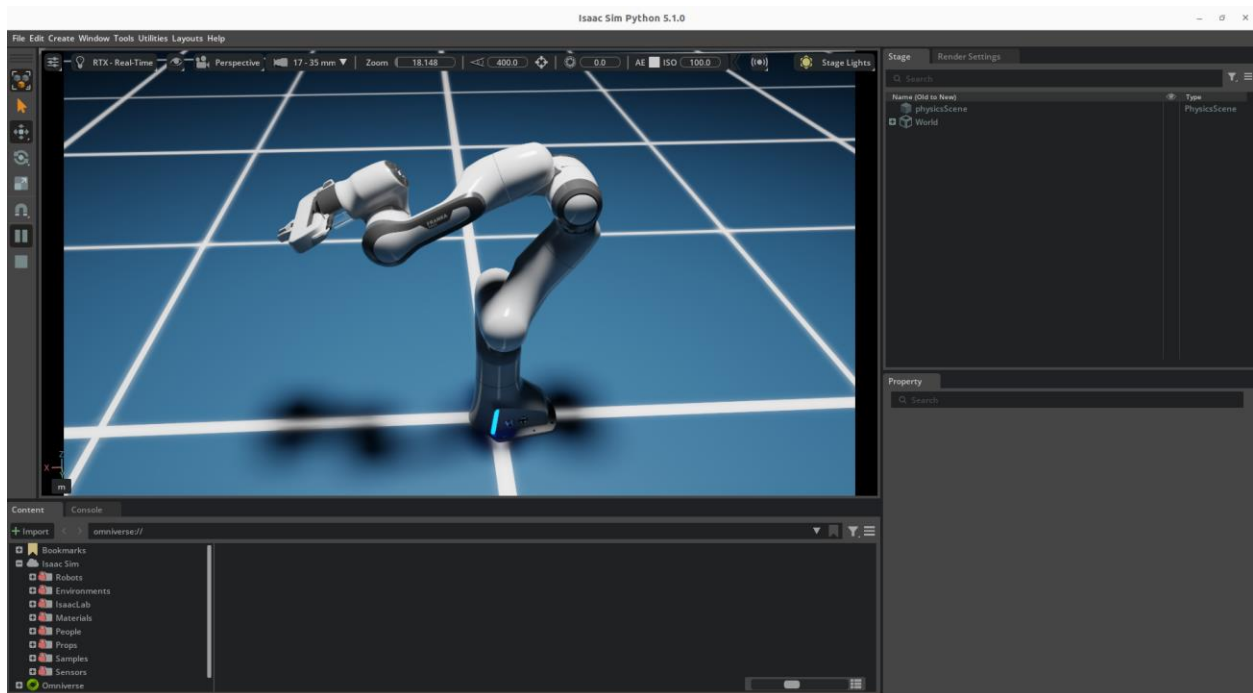


Figura 4.6: Simulador Isaac Sim

4.2.3. Plataforma software ROS 2

ROS 2¹ [33] es un middleware de código abierto diseñado para el desarrollo de aplicaciones robóticas modulares y escalables. Se compone de un conjunto de bibliotecas y herramientas que facilitan la comunicación entre los diferentes procesos del robot. Realmente es una herramienta muy potente y muy utilizada en robótica, pero en este caso se utilizará únicamente para la comunicación entre el simulador y el proceso de entrenamiento, ya que el control del robot se llevará a cabo a través de la API de la que dispone

¹ <https://docs.ros.org/en/foxy/index.html>

libfranka. Por ende, las características clave en lo que concierne a esta tarea son los siguientes:

- **Capacidad de tiempo real:** Gracias a su arquitectura DDS, ROS 2 permite una comunicación determinista, lo cual es imprescindible para bucles de control de alta frecuencia como el que se manejará (1kHz).
- **Conectividad con Isaac Sim:** El simulador está optimizado para la comunicación con este tipo de software.
- **Comunicación síncrona:** ROS 2 permite establecer un tiempo de reloj interno diferente al del sistema, para así permitir que tanto el simulador como el proceso de aprendizaje funcionen estén correctamente sincronizados, requisito indispensable para que la observación y la acción se corresponda adecuadamente.

4.2.4. Framework de RL: Stable Baselines 3

A la hora de realizar una tarea de aprendizaje por refuerzo no resulta práctico tener que implementar los algoritmos desde cero, teniendo que programar las ecuaciones matemáticas uno mismo, lo cual es propenso a errores. Es por eso por lo que en la práctica se suelen utilizar *frameworks*, es decir, un entorno de trabajo estandarizado para una tarea, diseñados específicamente para RL. En este caso se ha elegido *Stable Baselines* ² [34], debido a los diferentes factores:

- **Basado en PyTorch:** A diferencia de su predecesor y otros *frameworks* (como *OpenAI Baselines*) que utilizaban *TensorFlow*, SB3 utiliza *PyTorch*, lo que facilita la modificación del código.
- **Fiabilidad y limpieza:** Prioriza implementaciones estables y verificadas de algoritmos más estándar frente a posibles códigos más inestables de nuevos algoritmos que implementan otros *frameworks*.
- **Simplicidad:** Frente a otras opciones que añaden una sobrecarga de configuración inmensa, SB3 proporciona un uso más simplificado, con una curva de aprendizaje menor, aunque sigue permitiendo la optimización y modificación del código. Se puede cambiar el algoritmo utilizado con una simple línea de código.

4.2.5. Control visual: ViSP

ViSP³ (*Visual Servoing Platform*) [35] es una biblioteca de software de código abierto y multiplataforma desarrollada en C++ por el instituto de investigación Inria, y es el estándar académico e industrial para el desarrollo de aplicaciones de control visual. Su arquitectura modular permite implementar tareas de control visual de manera rápida y robusta, cubriendo:

- **Tracking:** Dispone de algoritmos en tiempo real para seguir características geométricas o basadas en modelo (MegaPose6D).
- **Control visual:** Implementa las leyes de control clásicas y calcula automáticamente las matrices de interacción, transformaciones de coordenadas y cinemática del robot.
- **Abstracción de hardware:** Se conecta fácilmente con cámaras y robots, incluyendo al Franka. Permite abstraerse del control directo del robot, pudiendo pasarle simplemente la velocidad generada por el algoritmo de control visual a una función de control. En el caso del Franka, a bajo nivel utiliza la propia librería de *libfranka*.

² <https://stable-baselines3.readthedocs.io/en/master/>

³ <https://visp.inria.fr/>

5. DESARROLLO

Esta es la sección principal de este TFG, en la cual se explicarán todos los procedimientos que se han seguido a lo largo del proyecto, cómo se han implementado los diferentes algoritmos con el objetivo de cumplir con los objetivos propuestos, y qué metodología se ha seguido para el trabajo. En el Anexo A: se presenta de manera esquemática la estructura de los códigos utilizados en el proyecto.

5.1. Objetivos

En este apartado se explicará cuáles son los objetivos que se quieren alcanzar en este Trabajo Fin de Grado, que definirán la metodología propuesta, y serán los indicadores de la evaluación y seguimiento de este proyecto.

El objetivo general del proyecto, definido en el Apartado 1.1, consiste en el control mediante impedancia de un brazo colaborativo con una cámara de eventos montada *eye-in-hand* para el seguimiento de un objeto a alta velocidad. Pero para garantizar el éxito de este proyecto es importante describir los objetivos específicos:

- **Implementación del control:** El primer objetivo requerido será, como es de esperar, la implementación de un código de control en impedancia para el robot FR3. Es un requisito totalmente indispensable para el proyecto, ya que será el que permita llevar la parte teórica a la práctica con el robot real. Se deberá demostrar un control básico del robot, con unas ganancias elegidas manualmente.
- **Obtención de ganancias dinámicas mediante RL:** Una vez se dispone de un control dinámico para el robot, será necesario llevar a cabo una tarea de aprendizaje por refuerzo para conseguir adaptar las ganancias de la ley de control a la tarea en tiempo real. El objetivo será demostrar que el algoritmo de RL es capaz de aprender este tipo de parámetros y que realmente benefician al comportamiento del robot frente a las otras implementaciones explicadas en el Apartado 3.1.
- **Control visual:** La última parte del proyecto, el control visual, será el encargado de realizar el seguimiento del objeto y controlar al robot en impedancia para que sea capaz de llegar a situarse encima de dicho objeto, con una velocidad de operación suficientemente alta para garantizar que realice la tarea satisfactoriamente.

5.2. Metodología

El plan de trabajo del proyecto consistirá, principalmente, en 3 apartados, que se corresponden con los objetivos planteados.

5.2.1. Control del robot

Aunque la finalidad de este apartado será la implementación del control en impedancia, también se desarrollarán diversos códigos de realización de trayectorias mediante control articular y cartesiano y un módulo de autocalibración, necesario para obtener la matriz de calibración de la cámara para el apartado de control visual. Además, estos pasos previos facilitarán la adaptación y comprensión de las librerías necesarias (*libfranka*) para el control del robot.

5.2.2. Tarea de aprendizaje por refuerzo

Dentro de este TFG, la tarea propuesta de RL se trata de la más innovadora y ambiciosa del proyecto, ya que se pretende que el robot sea capaz de aprender a generar dinámicamente las ganancias necesarias para el control. Al tratarse de una tarea que precisa de un robot que aprenda, se dividirá en 2 partes: simulación y

robot real.

El primer paso será adecuar el entorno de simulación para el aprendizaje: generar el entorno con el robot, trasladar el control en impedancia al simulador, desarrollar el proceso de RL por separado y comunicarlo correctamente con la simulación.

Una vez tengamos el entorno preparado, se diseñará el algoritmo de aprendizaje y se comenzará con la etapa de depuración. Se irá observando la evolución del aprendizaje, y planteando diferentes modificaciones para mejorar el algoritmo.

Si los resultados del entrenamiento en simulación cumplen los objetivos propuestos, se trasladará el aprendizaje al robot real, ya que habrá que lidiar con la brecha *sim-to-real*, es decir, la diferencia entre las dinámicas y condiciones simuladas y las reales.

5.2.3. Control visual

La última implementación será el control visual, donde seguiremos la misma metodología que en el caso del RL. Primero se desarrollará el algoritmo en simulación, utilizando una implementación de cero. Esto contribuirá a la comprensión y familiarización con los algoritmos de control visual, además que no se tendrá que utilizar ningún recurso de comunicación entre ViSP e Isaac Sim. Primero se implementará la detección del objeto dentro de la escena, utilizando una cámara simulada *eye-in-hand*. Una vez se disponga de una detección robusta del objeto se podrá proceder con el desarrollo del algoritmo de control visual, ya sea IBVS, PBVS o híbrido.

Cuando el control visual funcione correctamente en simulación, se trasladará al robot real mediante ViSP. Gracias a esta herramienta se podrá desarrollar el código más fácilmente, y realizar un control directo con el Franka. Además, también se podrán utilizar las librerías de tracking de las que dispone para refinar el seguimiento del objeto.

5.3. Control del robot

En este apartado se explicará el desarrollo del primer objetivo del proyecto, el control del robot. Se dividirá en las 4 implementaciones distintas citadas en la metodología del trabajo.

5.3.1. Control cinemático

En este apartado se agrupan tanto el control articular como el cartesiano, implementados a través de la librería del robot *libfranka*. Lo primero ha sido crear el código de control articular `moveJointTrajectory.cpp`. Este programa lee un archivo de texto con las diferentes configuraciones articulares que conforman la trayectoria, tanto en grados como en radianes (parámetro configurable en el código). Entonces se crea un bucle que va leyendo cada una de las configuraciones y moviendo al robot a una determinada velocidad mediante la clase `MotionGenerator()`, implementada en la propia librería del robot.

Por otro lado, tenemos el control en espacio cartesiano, el cual conlleva una lógica mayor. En este caso no se dispone de una clase a la que se le envíe la pose final del efector final deseada y automáticamente mueva al robot, sino que hay que diseñar un tipo de control diferente. Para esta implementación es importante recordar la frecuencia operacional del robot: 1kHz. Para mover al robot en el espacio cartesiano hay que trazar la trayectoria, desde la pose actual a la deseada, e ir generando las poses intermedias que la conforman.

En la librería `cartesianSave.cpp` se implementa este tipo de control, en la función `control_cart()`, que recibe como entrada la posición y orientación deseada, 2 parámetros de velocidad máxima para el control, un booleano para convertir o no de grados a radianes y otro booleano para guardar o no el estado del robot en un archivo json.

Primero se desacopla la traslación de la rotación, para tratar ambas tareas independientemente. Para que la rotación no tenga problemas a la hora del cálculo se transforma a formato *AngleAxis*, y se crea la rotación

deseada. Dentro del bucle de control, se calcula la distancia y rotación total a la meta, y se hace un control en aceleración, teniendo en cuenta el periodo de ejecución del bucle (1ms). Se va acelerando el robot poco a poco para evitar saltos o discontinuidades en la propia aceleración, que puedan hacer lanzar excepciones de seguridad del robot, calculando la aceleración conseguida mediante la derivada de la velocidad medida por el Franka. Cuando se detecta que el robot está llegando a la meta, se genera un cambio en la aceleración, frenando suavemente al robot antes de llegar a la pose deseada. Por último, para garantizar la suavidad de la trayectoria generada, se aplica un filtro de media móvil exponencial (EMA) con ganancia adaptativa, para que suavice el movimiento durante la traslación, pero se reduzca en la aproximación para asegurar precisión.

$$\mathbf{P}_f = \alpha(t) \cdot \mathbf{P}_{new} + (1 - \alpha(t)) \cdot \mathbf{P}_{prev}$$

Finalmente, el código `moveTrajectory.cpp` es el encargado de leer la trayectoria de poses que va a seguir el robot y hacer el bucle de comportamiento, encargado de llamar a la función `control_cart()` para mover al robot. Este código ha sido necesario para la obtención de un dataset para la parte de visión del proyecto.

5.3.2. Módulo de autocalibración

Dentro del proyecto también se tuvo que implementar un módulo de autocalibración, que consistía en que lanzando un único código se llevase a cabo la calibración de la cámara montada en el efector final del robot. El encargado de la calibración lo único que tiene que hacer es situar una hoja con un tablero de ajedrez impreso en el entorno (se ha dejado indicado dónde situar dicha hoja). Este módulo contaba de 2 partes, la obtención de los parámetros intrínsecos y extrínsecos de la cámara, y el movimiento del robot para obtener las imágenes necesarias para ello. El alcance de este TFG se limita a la integración del control del proyecto, la parte de la percepción visual, encargada en este caso de la calibración, ha sido desarrollado por mi compañero Gabriel Barrigón Gómez [36].

El código `calibracion_joint.cpp` mueve mediante control en espacio articular al robot a un conjunto de 12 poses diferentes, desde las que va a tomar las imágenes necesarias para la calibración. Para ello, se ha calculado un conjunto de 12 puntos que mantienen la misma distancia al centro del *chessboard*, en diferentes puntos de vista, generando 2 órbitas alrededor del tablero. Gracias a esto, se puede cerciorar de que las imágenes conseguidas son las óptimas para hallar los parámetros de la cámara. Por otro lado, el control se ha realizado en espacio articular ya que es más robusto ante configuraciones singulares, y debido a su repetibilidad, es la opción más segura y eficiente. En la **Figura 5.1** se puede observar el entorno para la calibración.

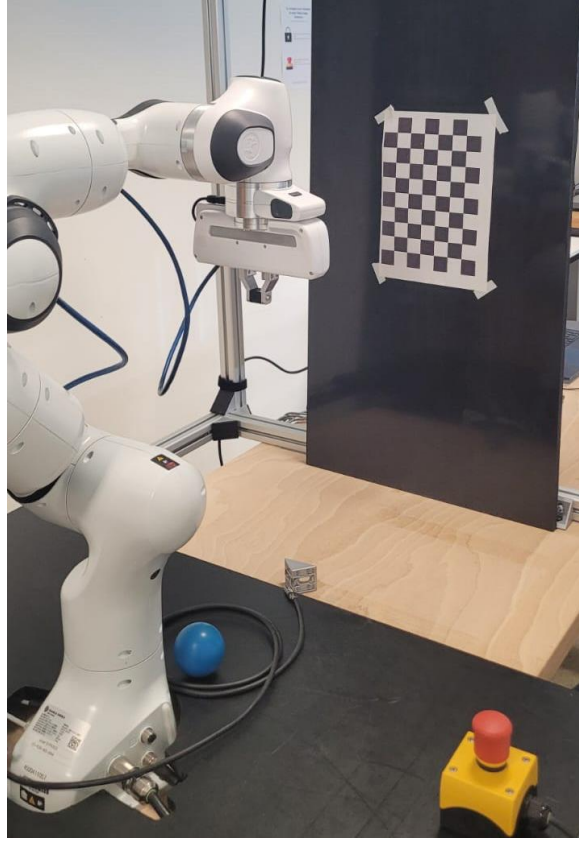


Figura 5.1: Entorno de trabajo para la autocalibración

5.3.3. Control en impedancia

Antes de pasar a la parte de implementación, será necesario explicar cómo se obtiene la ley de control, y de dónde aparecen los términos que se van a estar utilizando. Este desarrollo toma como base teórica el capítulo de control en impedancia del libro de Bruno Siciliano et al. [37].

La dinámica de un manipulador robótico de n grados de libertad (7 en el caso del Franka FR3) describe la relación entre las fuerzas/pares aplicadas a las articulaciones y el movimiento resultante del robot. Utilizando el formalismo de Euler-Lagrange, la ecuación de movimiento en el espacio articular se expresa como:

$$M(q) \ddot{q} + C(q, \dot{q}) + g(q) = \tau$$

Este es el modelo cinemático del robot. Se consideran las dinámicas que se aplican sobre el robot, es decir, la masa inercial M , el efecto de Coriolis C y la gravedad g , y el conjunto de estas ejerce un determinado torque τ sobre las articulaciones del robot. Ahora bien, el control en impedancia se basa en un comportamiento masa-muelle-amortiguador, que se puede modelar a partir de la siguiente ecuación:

$$M \ddot{q} + D \dot{q} + K (q - q_d) = 0$$

En este caso se introducen los términos de *damping* y *stiffness*, D y K , ya que se intenta modelar la física de un muelle y de un amortiguador. Además, como lo que pretende este modelo es alcanzar una determinada configuración articular meta, q_d , se añade el error articular. Por ende, ahora para conseguir la dinámica deseada hay que definir la aceleración. Con esto, se consigue crear un controlador PD en espacio de aceleración:

$$\ddot{q}_{\text{ref}} = K (q_d - q) - D \dot{q}$$

Esta es la aceleración que se necesita que siga el controlador para alcanzar correctamente una pose articular deseada. Lo único que queda por hacer es sustituir esta aceleración en el modelo cinemático del robot, con lo que conseguimos la ley de control en impedancia articular con compensación dinámica completa:

$$\tau = M(q) (K (q_d - q) - D \dot{q}) + C(q, \dot{q}) + G(q)$$

Esta será la ley de control que se implementará para el aprendizaje, ya que el espacio articular es más seguro ante singularidades que el espacio cartesiano, y al no tener que calcular matrices jacobianas es más directo. Pero a la hora de realizar un ejemplo de control en impedancia, sobre todo adaptando manualmente las ganancias de rigidez y amortiguamiento, es más eficiente utilizar la ley de control en impedancia cartesiana. Para conseguirla simplemente hay que adaptar la anterior al espacio cartesiano utilizando la matriz jacobiana del robot. Teniendo en cuenta que la nueva matriz inercial, ahora en espacio cartesiano, es la siguiente:

$$M_x = (J M^{-1} J^T)^{-1}$$

La ley de control en impedancia para el espacio cartesiano resulta en la siguiente:

$$\tau = J^T M_x^{-1} (K_x (x_d - x) + D_x \dot{x}) + C(q, \dot{q}) + g(q)$$

Ahora que ya están formalizadas las leyes de control que se van a seguir en este proyecto, se puede pasar a la implementación del código de control en impedancia para el robot real. Para esto, la librería de *libfranka* dispone de una entrada de control para el control en impedancia, donde se le puede pasar los torques calculados; pero necesita también un control de pose cartesiana para conseguir la pose deseada, del mismo modo que se ha explicado en el anterior punto. De este modo, lo que hace el controlador, es utilizar esta pose generada como referencia para calcular los nuevos torques, tal y como se muestra en la ley de control cartesiana. Así, obtenemos los torques para mover al robot a la pose inmediatamente posterior, teniendo en cuenta el periodo de 1ms en el bucle de control. Esta implementación está en el código `control_impedancia.cpp`, donde las ganancias utilizadas (K y D) son generadas manualmente, mediante prueba y error.

5.4. Aprendizaje de los parámetros mediante RL

Esta sección se dedicará a la implementación de un algoritmo de RL para el aprendizaje de los parámetros de rigidez y amortiguamiento del control en impedancia. Primero se explicará el desarrollo del *script* de simulación, que incluye la traslación del control en impedancia al simulador y la definición del entorno. A continuación, se planteará la tarea inicial de aprendizaje por refuerzo, detallando la primera *reward function*, el algoritmo de RL implementado y el entorno de la tarea. Una vez sea aclarado este segundo paso, se explicará la comunicación mediante ROS 2 entre el simulador y el agente. Finalmente, se podrá explicar el proceso de depuración del aprendizaje, es decir, qué problemas se han ido encontrando, cómo se han solucionado y las propuestas desarrolladas en este TFG para lograr los objetivos del proyecto.

5.4.1. Entorno de simulación en Isaac Sim

Para desarrollar el código de simulación hay que tener en mente el objetivo propuesto, el entrenamiento de un algoritmo de Deep RL, por ende, los recursos computacionales del ordenador serán una variable a tener en cuenta. Por eso mismo, lo primero es lograr que el simulador se lance de manera *headless standalone*. Cuando se habla de lanzar un simulador *standalone* quiere decir que se puede lanzar de manera independiente, que el software se puede lanzar “por sí solo”. En este caso, lanzar un código de Isaac Sim *standalone* se refiere a ejecutar un código Python que inicie el simulador y se ejecute automáticamente el *script* de control. Por otro lado, lanzarlo de manera *headless* significa que no se mostrará el entorno gráfico del simulador, simplemente se estarán calculando todas las dinámicas y el estado visual del entorno necesario, pero sin mostrarlo por

pantalla. Esto ahorrará considerablemente los recursos computacionales utilizados por el simulador, ya que, además, Isaac Sim, al ser un simulador con unas características visuales muy potentes, también es computacionalmente muy exigente.

El código está organizado en clases para mayor limpieza de código e independencia de la simulación y el control en impedancia (y posteriormente la comunicación con el agente). En la clase `FrankaImpedanceSimulation` se declara el entorno de simulación en la función `setup()`, creando el mundo y el robot Franka Research 3, y se inicia la simulación. En la función `initialize()` se controla que el simulador se inicie correctamente, y avisa al bucle principal, el `on_update()`, de que la simulación ha empezado correctamente. Esta función es la encargada de leer el estado del robot y enviarlo al controlador, a la clase `JointImpedanceController`.

Para la clase del controlador generamos la función que va a implementar la ley del control en impedancia articular, descrita en el anterior apartado. Necesitará la dinámica actual del robot, por lo que se utilizan las propias funciones del simulador, para conocer la masa inercial, el efecto Coriolis y la gravedad de cada una de las articulaciones. A partir de estos datos, la configuración articular actual, la deseada, y la velocidad articular actual, calcula los torques que hay que aplicar al robot para alcanzar la pose deseada. Inicialmente, y para observar que la implementación es correcta, las ganancias K y D serán estáticas y declaradas con valores estándar.

5.4.2. Elección del algoritmo: Soft Actor Critic

La elección del algoritmo de RL es un paso de vital importancia para el aprendizaje, ya que el éxito del entrenamiento puede depender del algoritmo implementado y, además, definirá los parámetros de entrenamiento, la función de recompensa, cómo se aborda el dilema exploración-explotación y la naturaleza de el espacio de acción y observación. Es por este último punto por el que se empezará a definir el entorno.

El espacio de observación se compondrá, inicialmente, por 2 vectores de 7 componentes cada uno: q_{error} y $\dot{q}$, es decir, el error articular ($q_d - q$) y la velocidad articular. Estos serán los parámetros que reciba la red neuronal como información. El espacio de acción, es decir, lo que tiene que devolver la red, lo formarán también 2 vectores de 7 componentes: K (*stiffness*) y D (*damping*). Son vectores de 7 componentes ya que, como el propósito del aprendizaje es generar las ganancias óptimas para el correcto comportamiento del robot, es necesario otorgar de manera individual las ganancias a cada articulación del robot.

Una vez definidos los espacios de observación y de acción, es necesario seleccionar un algoritmo de aprendizaje por refuerzo adecuado para espacios continuos. Tanto el error articular como la velocidad son magnitudes que no se pueden discretizar para esta tarea, ya que se necesita conocer el valor concreto. Por otro lado, como se buscan las ganancias óptimas que se adapten a la dinámica actual del sistema, tampoco se pueden discretizar. Es por eso por lo que ambos espacios son continuos.

Existen diferentes algoritmos de Deep RL que utilicen espacios continuos, y en este Trabajo Fin de Grado se ha estudiado el uso de 3 distintos: DDPG [13], TD3 [38] y SAC [39] [40]. Todos ellos están basados en una arquitectura común, el uso de redes *actor* y *critic*. La red *actor* es la encargada de aprender la política a seguir por el agente, en cambio, la red *critic* se encarga de la evaluación de la política que se está aprendiendo. En mayor profundidad, el *critic* genera un Q-valor a partir de un par estado-acción (s,a), y esta valoración es la que se utiliza en la red *actor* para calcular su *loss function*, y así lograr que la red “aprenda” gracias al descenso del gradiente (explicado en el Apartado 2.4.1.4). Por otro lado, la red *actor* tendrá como entrada el estado s y como salida la acción a . En el caso de ser una política estocástica, en vez de devolver únicamente la acción a , devolvería 2 parámetros diferentes: la media de la acción (μ) y su desviación típica (σ).

1. **DDPG (Deep Deterministic Policy Gradient):** Es el algoritmo fundacional para espacios de acción continuos. Adapta la metodología de DQN (que utiliza un espacio de acción discreto) utilizando redes *actor* y *critic*. Esta implementación, aunque dispone de un uso continuado a día de hoy, suele tener inestabilidad y sensibilidad frente a los hiperparámetros elegidos. Además, sufre de sobreestimación del Q-valor de la red *critic* (cree que la acción elegida es mejor de lo que es realmente), lo que conlleva a que la política generada por el *actor* sea subóptima.
2. **TD3 (Twin Delayed DDPG):** Este algoritmo es directamente una propuesta para solucionar los

problemas de DDPG, que introduce las siguientes 3 mejoras:

- i. **Twin (Gemelos):** Utiliza 2 redes neuronales para el *critic* en vez de una, y se queda con el valor mínimo de los 2 Q-valores generados. De este modo, elimina la sobreestimación, teniendo un comportamiento más pesimista.
 - ii. **Delayed (Retardado):** Actualiza la red del *actor* a menos frecuencia que la del *critic* para reducir la varianza.
 - iii. **Target noise:** Añade ruido a la acción objetivo durante el entrenamiento para suavizar la política y evitar que se "sobreajuste" a picos estrechos de error.
3. **SAC (Soft Actor Critic):** Mientras que TD3 es una mejora de DDPG, SAC es una mejora del algoritmo A2C [12] y, por ende, en lugar de seguir una política determinista como los anteriores, SAC utiliza una política estocástica. Esto significa que no devuelve una acción fija, sino que devuelve una distribución gaussiana, para no solo decir cuál es la mejor acción a seguir, sino cuán seguro está de ello. Por otra parte, SAC no solo busca maximizar la recompensa, sino que también la entropía. Esta es la gran novedad que presenta este algoritmo.

La entropía incentiva al agente a explorar más y a no converger prematuramente, solucionando a su vez el dilema exploración-explotación (cuánto hay que explorar y cuánto hay que explotar el conocimiento aprendido). Además, al haber "aprendido" con entropía, tras finalizar el entrenamiento y utilizar la política creada por el *actor*, es más robusto al ruido que los otros algoritmos, ya que en el entrenamiento tenía esta componente de aleatoriedad. Por otra parte, se trata de un algoritmo *off-policy*, lo que significa que utiliza una política para el comportamiento en el entrenamiento (*behavior policy*) que es distinta a la política que aprende (*target policy*). En lo que deriva esto es en que es más eficiente en el número de muestras, ya que aprende más con menos datos. Además, también utiliza 2 redes *critic* (*Double Q-learning*), lo que desemboca en un aprendizaje más estable.

En la **Figura 5.2**, proveniente del artículo de Haarnoja et al. [39], se muestran los resultados de varios entrenamientos utilizando diferentes algoritmos (SAC, DDPG, PPO, SQL y TD3), donde se demuestra que, en general, se han obtenido mejores resultados utilizando SAC. En la **Figura 5.3** se muestra la estructura del algoritmo de SAC.

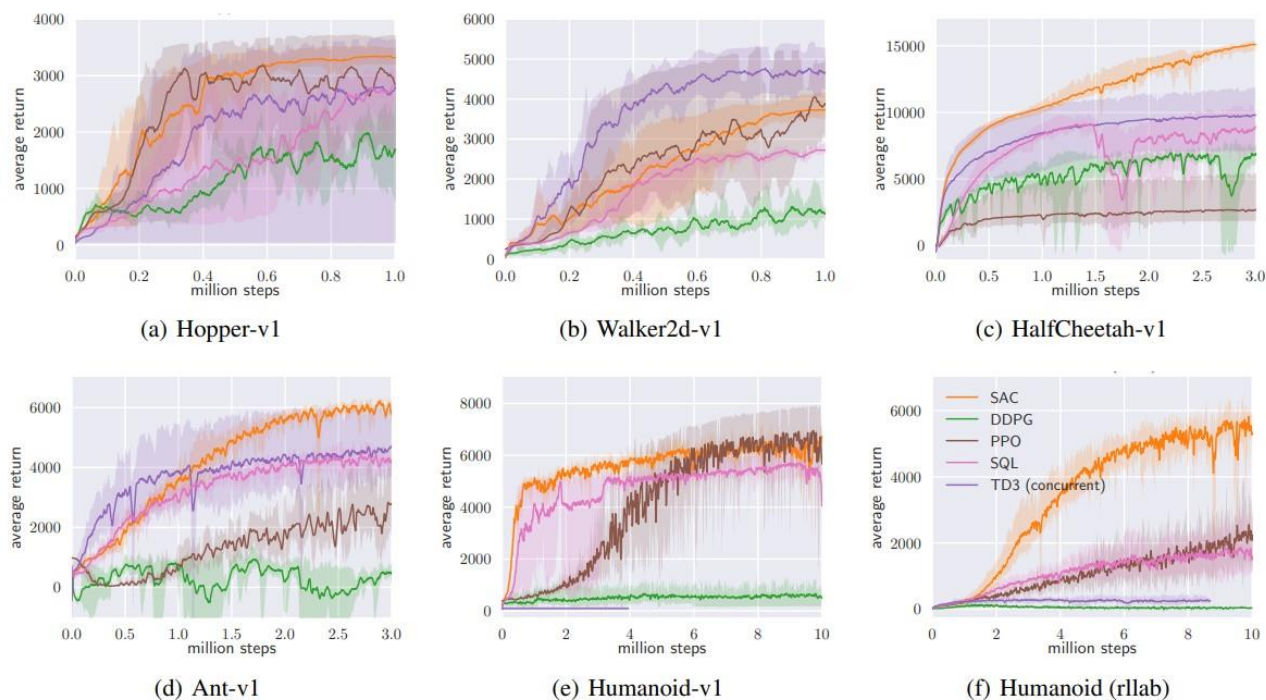


Figura 5.2: Resultados de entrenamiento de diferentes algoritmos de Deep RL

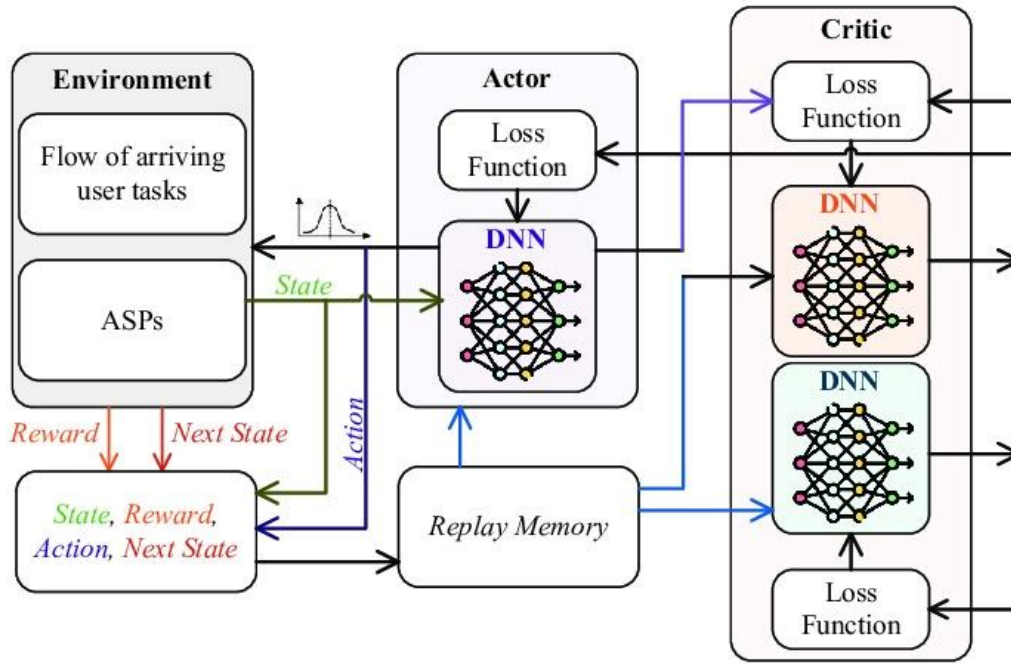


Figura 5.3: Estructura del algoritmo de Deep RL Soft Actor Critic [41]

5.4.3. Implementación del agente

Ahora que ya se ha explicado el algoritmo que se va a utilizar, se puede proceder con la implementación del agente, utilizando el *framework* de Deep RL de SB3. Este entorno de trabajo permite abstraerse de la implementación de la estructura y matemática del algoritmo, de crear el *loss function*, el descenso del gradiente, las diferentes redes neuronales, el *buffer* de datos, etc. SB3 ofrece llevar a cabo la tarea de aprendizaje por refuerzo de manera más simple y directa, configurando el entorno y los hiperparámetros del algoritmo y las redes. Dicho esto, se comienza con la configuración de la primera implementación del agente.

Como se ha explicado antes, la observación del agente es un vector de 14 valores, compuesto por el error y la velocidad angulares de las articulaciones del robot, pero el estado del robot dispone de una mayor información. La primera función recompensa se trataba de una recompensa densa (*dense reward*), es decir, en cada *step* del entrenamiento se calcula una nueva recompensa que se utiliza para conseguir que la red neuronal aprenda. Se trataba de la siguiente suma ponderada:

$$reward = - (a \cdot |e_{ang}| + b \cdot \ddot{q}_{term} + c \cdot \dot{q}_{term} + d \cdot \Delta u)$$

Esta *reward function* estaba compuesta por los siguientes parámetros:

- **Error angular:** El valor absoluto del error angular, para que la red aprenda a llegar a la pose deseada.
- **Término de jerk:** El *jerk* es la derivada de la aceleración angular, y sirve para medir las vibraciones que sufre el brazo robot. Si el *jerk* es muy alto significa que el robot ha tenido una aceleración muy brusca, y esto es algo muy importante a considerar ya que, no solo puede ser peligroso para el robot, sino que si supera un cierto valor el controlador a bajo nivel del Franka lanzará una excepción de seguridad para detener el robot. El término que se ha utilizado es un término logarítmico, ya que puede llegar a valores muy altos, del orden de $10.000 \frac{rad}{s^3}$. De este modo, y haciendo una normalización después, aunque la oscilación sea mucho menor, de $100 \frac{rad}{s^3}$, se sigue teniendo en cuenta y no desaparece por ser demasiado pequeño.
- **Término de velocidad:** Se requiere incentivar al agente a que la velocidad no decaiga y tarde en llegar a la pose deseada, haciendo que el movimiento sea demasiado “blando” (reduciendo mucho las ganancias), por ende, se añade al *reward* un término de velocidad para que, según la distancia a la que esté de la meta, recompensar al agente por generar una velocidad acorde.
- **Incremento de la acción:** El espacio de acción es bastante amplio, por lo que durante el entrenamiento

el agente puede generar unas ganancias muy bajas y en la siguiente iteración unas muy altas. Este cambio tan brusco en las ganancias de control provoca inestabilidad en el sistema, por lo que es necesario reducir estos cambios tan bruscos, introduciendo en la recompensa una componente de diferencia entre la acción anterior y la nueva generada por el agente.

Por otra parte, si el agente es capaz de alcanzar la meta con unas condiciones de oscilación aceptables, se finalizará el episodio con una recompensa positiva. En lo que concierne a los hiperparámetros del agente, se utilizarán unos parámetros estándar, ya que luego al hacer pruebas se procederá al refinamiento de los mismos. Además, la entropía del entrenamiento será aprendida por el propio algoritmo de SAC con las condiciones normales.

5.4.4. Comunicación entre el simulador y el agente

Ahora que ya están definidos los códigos del simulador y el agente de RL hay que explicar la comunicación entre ambos procesos, y qué cambios hay que hacer para ello. El principal propósito es que al agente le llegue la observación del entorno, genere una acción a ejecutar, y el *script* de simulación utilice dicha acción para controlar al robot simulado.

Se crean 2 nodos de ROS 2, uno para el agente y otro para el código de simulación. En simulación se crea una nueva clase `ROS2Bridge`, que controlará el flujo de comunicación dentro del simulador. Esta clase lanzará los *callbacks* para escuchar al agente y creará los publicadores para mandarle información del estado. Además, se añadirá la función `apply_pending_updates()` dentro del controlador para actualizar adecuadamente los valores de las ganancias y la configuración articular deseada antes de calcular los nuevos torques.

Por otro lado, un requisito muy importante para el correcto funcionamiento del entrenamiento es cerciorarse de que las acciones generadas se corresponden con las observaciones. Por eso, es de vital importancia que, tanto el simulador como el agente, funcionen a la misma frecuencia. Para ello, se establece mediante el *topic* de ROS 2 `/clock` el tiempo de reloj común, enviando este mensaje a 1kHz de frecuencia (la frecuencia de funcionamiento deseada) y en otro hilo aparte, para que no acapare los recursos de la ejecución del simulador.

En el caso del agente se definirá la clase `ROSEnv`, que hereda de la clase del entorno de la librería `gymnasium`. En esta clase se crea el nodo de ROS 2 y, al mismo tiempo, el entorno de `gymnasium` que necesita SB3, donde se crean los suscriptores y publicadores para recibir la observación del simulador y enviar la acción correspondiente.

La tarea que se está implementando es de aprendizaje, y los robots, al igual que los humanos, durante el aprendizaje van a tener errores; por eso, el aprendizaje tiene que ser robusto ante este tipo de situaciones. Para ello, el simulador observa que el robot no haya colisionado, entrado en una configuración singular o está intentando forzar demasiado las articulaciones. En ese caso, envía un mensaje de *reset* al agente, el cual termina el episodio con una recompensa muy negativa, para que el agente aprenda que ese comportamiento es erróneo, y espera a que el simulador limpie la escena y vuelva a generar el mundo correctamente. Además, el simulador también limpia la escena y libera memoria cada cierto número de episodios, para no saturar la gráfica y que se pueda ejecutar el aprendizaje no sólo horas, sino días o incluso semanas. En este caso, envía otra señal de *reset* al agente, pero esta vez no tiene que aportar una recompensa negativa. Además, al hacerse al finalizar el episodio, no pierde ningún progreso.

5.4.5. Desarrollo del aprendizaje

Una vez el agente está implementado y es capaz de sincronizarse correctamente con el simulador comienza la etapa más importante del entrenamiento, modificar y adaptar el agente para que sea capaz de aprender la tarea impuesta. Para esto, se propuso un aprendizaje por curriculum, es decir, en vez de forzar al agente a aprender el comportamiento de manera directa, hacerlo más parecido a como lo haría un humano, partiendo de un aprendizaje base e ir incrementando gradualmente la dificultad. Por ende, en vez de mover todas las articulaciones para alcanzar una nueva pose deseada, se propone partir de la pose de reposo (ver **Figura 5.4**) y mover únicamente una articulación con una rotación aleatoria, no muy grande para evitar posibles colisiones. Una vez el agente haya aprendido lo suficiente en cada articulación, se pasaría a ir utilizando un mayor

número de articulaciones al mismo tiempo.

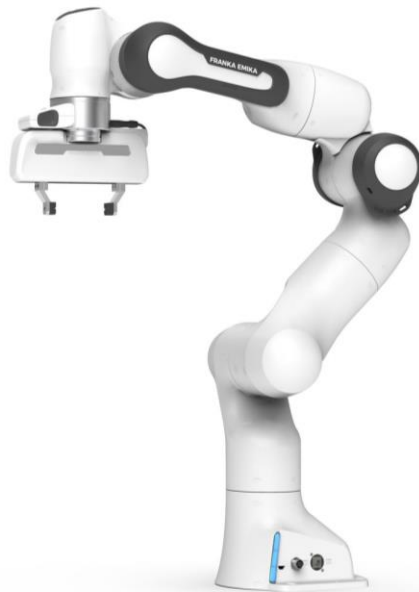


Figura 5.4: Franka Research 3 en pose de reposo (*home*)

Por otro lado, se observaba una gran inestabilidad en el *critic loss* del aprendizaje; por ende, se propuso una mejora: normalizar tanto el *reward* como el espacio de acción y observación. Este método, utilizado mediante la clase *VecNormalize* de SB3 para configurar el entorno, redujo drásticamente el crecimiento y oscilaciones en la función de pérdida de la red *critic*, proporcionando una mayor estabilidad al entrenamiento, ya que al explotar de tal manera el *critic loss*, la política generada no se evaluaba correctamente.

Al principio se utilizó la entropía por defecto, es decir, el algoritmo SAC creaba una propia función de pérdida para “aprender” a configurar la entropía del entrenamiento. El problema con esta implementación es que explotaba demasiado rápido, sobre todo teniendo en cuenta el tiempo necesario de entrenamiento, por lo que se optó en primera instancia por configurar manualmente este valor de entropía. Ahora el problema venía con la dificultad para adaptar la entropía al estado del entrenamiento, si se dejaba un valor fijo establecido no era óptimo para todo el transcurso del entrenamiento, ya que al principio tiene que explorar suficiente, pero luego tiene que ser capaz de explotar el conocimiento, aunque siga aprendiendo. Por ende, se optó por crear una función para ir bajando poco a poco, cada cierto número de iteraciones, la entropía. El problema de esta implementación es que era “ciega”, es decir, simplemente depende del tiempo, y no de la progresión del entrenamiento. Finalmente, se optó por modificar directamente la *loss function* de la entropía, para que el algoritmo volviese a aprender a generar la entropía, pero esta vez, alterando una serie de parámetros, se pudo evitar la explosión de esta.

Por último, se decidió modificar el espacio de observación, para que en vez de utilizar el q_{error} se desglosase en q_{actual} y q_{meta} . Esta modificación se llevó a cabo porque no solo es importante conocer la rotación que tiene que ejercer cada motor para alcanzar la pose deseada, sino que también influye cómo está configurado el robot. Si el robot está completamente extendido hacia el frente va a tener que ejercer un mayor torque que si está extendido hacia arriba, ya que tendrá que compensar la gravedad ejercida sobre la estructura, la inercia generada y el efecto de Coriolis.

5.4.6. Problema con controlador y redefinición de la tarea

En los resultados del aprendizaje se observaba un problema constante, las ganancias necesarias para conseguir alcanzar la meta deseada debían ser demasiado grandes. Además, existía un *offset* sobre todo en la sexta articulación del robot (la muñeca), haciendo que la mano mirase hacia el frente, en vez de hacia abajo. Esto hacía que al robot se le dificultase mucho alcanzar la meta, e incluso que con determinados umbrales para generar las ganancias no fuese capaz directamente. Se estuvo investigando esta problemática y se encontró la

solución en el código de simulación del robot.

El problema venía de una mala implementación del código de Isaac Sim. Para generar el movimiento en la versión utilizada (5.1.0) se utiliza la clase `ArticulationAction`, a la cual se le puede pasar como argumento una configuración articular, velocidad articular o los torques generados por el control en impedancia. Lo que pasa es que, tanto pasando q como $\dot{q}$, realmente a bajo nivel se lleva a cabo un control en impedancia. El problema es que ese controlador sigue operativo, intentando mover al robot a la configuración articular $q = \vec{0}$ (ver **Figura 2.2**). Es por eso por lo que el controlador necesitaba unas ganancias muy grandes para alcanzar la pose deseada, ya que realmente había un segundo controlador que tenía que contrarrestar.

Al eliminar este segundo controlador se hizo una prueba, y se observó que ahora el robot sí era capaz de alcanzar la pose deseada sin ningún problema, e incluso el *jerk* había bajado, aunque seguía siendo alto por lo general. Por ende, en este momento surgió la necesidad de alterar la tarea, ya que ahora no era demasiado problema alcanzar la pose.

El enfoque del aprendizaje por curriculum se cambió por 2 motivos. Por un lado, se observó que el control sobre unas articulaciones afectaba al resto, por lo que no se podía realizar un control en las articulaciones de manera independiente, sino que había que utilizarlas todas al mismo tiempo. Esto antes no era viable, pero al solucionar el problema del controlador a bajo nivel, ahora el agente era capaz de llegar a la pose deseada, aunque tuviese que controlarlas todas. Por ende, lo que se propuso fue generar un espacio de trabajo cartesiano, y generar la pose deseada como pose cartesiana, en vez de articular. Se establecieron diferentes niveles de dificultad, y a cada cual se iba aumentando este espacio de trabajo, tanto en posición como en orientación posible teniendo como referencia la mano perpendicular al plano. Entonces se generaba una pose aleatoria dentro de este espacio, y que estuviese a una determinada distancia mínima de la última pose generada (si estaba muy cerca el episodio no tenía sentido). Además, se utilizó este nivel de dificultad para alterar la entropía del aprendizaje, ya que cuando el espacio de trabajo aumentaba, se daba por supuesto que el robot debería explorar más.

Por otra parte, se decidió también modificar la función *reward*. Ahora, sería una función de tipo *sparse*, es decir, que el agente en lugar de recibir una recompensa total calculada en cada iteración devolvería la recompensa únicamente al finalizar el episodio. Además, en vez componerse por una suma ponderada, será una sucesión de penalizaciones por:

- **Iteración:** Al penalizar por defecto en cada iteración, se premia que el agente termine lo antes posible.
- **Exceso de *jerk*:** Es necesario que el agente ajuste las ganancias para que el robot no sufra vibraciones.
- **Torques generados:** Se calculan los torques generados por el control en impedancia para que, si superan los límites del robot, se penaliza y el agente aprenda a no generar torques demasiado altos.
- **Velocidad baja:** Se necesita que el robot sea rápido en su ejecución.

Por otra parte, ahora el episodio termina cuando el agente llega a la meta, recibiendo una recompensa positiva. Además, el umbral que define que el robot ha llegado a la meta varía según el nivel de dificultad.

5.4.7. Nuevo enfoque: perfil trapezoidal

Al solucionar el problema con el controlador a bajo nivel el robot llega sin problema a las poses propuestas, pero el problema ahora es que sigue generando *jerk*. El agente no era capaz de aprender a reducir estas vibraciones, por lo que se investigó que era lo que estaba sucediendo realmente. Se propuso modificar las ganancias cada 200 iteraciones, en vez de en cada iteración, y entonces el *jerk* descendió en gran medida. Se observó que una causa importante a la hora de la generación de *jerk* era la oscilación de las ganancias. Al variar demasiado rápido las ganancias utilizadas en el control, el robot generaba unas vibraciones muy altas, por lo que, al reducir la frecuencia de actualización de las ganancias, estas vibraciones menguaban. El problema es que esto repercutía en varias cosas.

Por un lado, el tiempo de entrenamiento se incrementaba 200 veces, ya que en lugar de actualizar la red cada iteración ahora se hacía cada 200, lo cual provocaba que el tiempo necesario para conseguir resultados no fuese admisible. Por otro lado, al variar tan poco las ganancias, se perdía de vista un poco el objetivo de la

implementación del aprendizaje por refuerzo profundo, que era conseguir las ganancias de manera dinámica y adaptativa al movimiento generado. Además, se observó que la generación de *jerk* no sólo dependía de la frecuencia con la que se generaban nuevas ganancias, sino que también del incremento entre la ganancia actual y la siguiente, generando picos de *jerk* muy altos.

Con estas premisas, se propuso un nuevo enfoque de la tarea, que la generación de las ganancias siguiese un perfil trapezoidal, como el mostrado en la **Figura 5.5**. Esto significa que tanto K como D empiezan con valores muy bajos, y se van incrementando poco a poco (ya que con valores bajos de ganancia el robot no tiene suficiente fuerza y le cuesta moverse). Luego, llegan a un valor máximo predefinido, y cuando se acerca el robot a la meta, reducen las ganancias poco a poco para que cuando llegue, se pare, y cambie la trayectoria, no genere *jerk*. Para ello, primero se demostró que la teoría era cierta, haciendo un programa que generase las ganancias siguiendo este tipo de perfil trapezoidal, observando que el *jerk* generado en toda la trayectoria era muy reducido y totalmente asumible por el robot (ver en resultados). Una vez demostrado, se puso en práctica, intentando incitar al agente a que aprendiese este tipo de comportamiento.

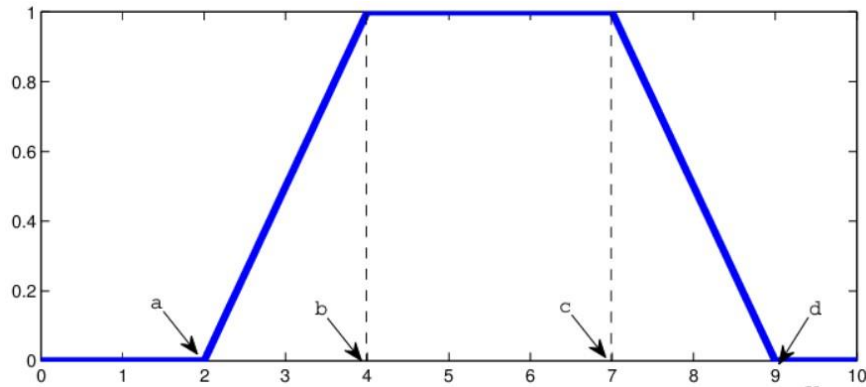


Figura 5.5: Perfil trapezoidal

Para ello, se comenzó añadiendo 2 nuevas penalizaciones al *reward*. La primera se trata de generar 2 intervalos de ganancias posibles, un intervalo de ganancias bajas para la subida y bajada del trapezoide, y otro intervalo más alto para la parte estacionaria. Si las ganancias generadas no entran dentro del intervalo específico, se penaliza al agente. Por otra parte, para la segunda penalización se tuvo que modificar el espacio de observación, añadiendo la última ganancia generada (con la K era suficiente). Si había demasiada diferencia entre la ganancia actual y la anterior, se penalizaba al agente, para intentar que aprendiese a aumentar las ganancias paulatinamente.

5.4.8. Control basado en aceleración

Durante el entrenamiento del agente, se intentó buscar otro tipo de opciones para el control en impedancia, en la procura de encontrar otros tipos de enfoque. Se pensó en cómo funciona realmente el control en impedancia en el robot real, el cual necesita a mayores un control de la trayectoria, para, en vez de enviarle directamente la pose final deseada, descomponer la trayectoria en una sucesión de poses muy cercanas entre sí, haciendo un control de la aceleración. Se implementó este sistema para el cálculo de los torques de la ley de control en el simulador, y se observó unos resultados muy curiosos. Efectivamente ahora el *jerk* generado era mucho menor, y totalmente admisible por el robot, pero traía consigo un nuevo problema. Ahora, al modificar las ganancias de K y D no se apreciaba ninguna diferencia en la salida de los torques generados o la velocidad articular. Además, en el *jerk* tampoco se apreciaba gran diferencia entre utilizar una selección de ganancias u otra, inclusive siendo igual en valores altos.

5.5. Control visual

La metodología de la implementación del control visual ha sido la misma que en el caso del algoritmo de Deep RL, primero desarrollar la tarea en simulación, para luego poder pasar al robot real. Para esta segunda implementación se utilizaron las cámaras mencionadas en el Apartado 4.1.2, comenzando por la RealSense

D405 y finalizando la implementación final con la cámara de eventos EVK4.

5.5.1. Simulación

Para la implementación de la tarea de control visual en simulación se construyó el código en 2 fases, un primer acercamiento con una implementación muy sencilla, sin utilizar matriz de interacción, y luego ya se implementó el algoritmo de IBVS. Lo primero de todo es detectar el objeto que tiene que seguir el robot. El objetivo del proyecto era utilizar una pelota azul, ya que la morfología de la misma facilita un movimiento rápido. Por ende, y partiendo de la premisa de que el entorno de trabajo es una tabla de color negro, se puede hacer una simple detección por RGB. Se busca en la imagen un objeto de color azul, y se encierra en un *bounding box*, para de este modo obtener las 4 esquinas del objeto.

Para la primera implementación de prueba, en vez de utilizar las 4 esquinas del *bounding box* se utiliza el centro de la caja, y mediante las matrices de transformación homogénea para pasar de la información visual al sistema de referencia del *end effector*, se conoce cómo afecta la velocidad cartesiana al error visual. Para la corrección en profundidad, se parte de la premisa de que se conoce el tamaño exacto del objeto, por ende, se puede estimar la profundidad al objeto mediante el cálculo del área del *bounding box* actual, y el área deseada. Para mejorar el comportamiento, se implementó también un controlador PD, para evitar que el *end effector* oscilase, se pasase de largo y perdiese la vista de la pelota, e intentar que alcanzase cierta velocidad. Además, también se llevó a cabo una parada por zona muerta, para que cuando el robot llegase a la meta no estuviese oscilando para intentar corregir un error muy pequeño.

En la segunda implementación ya se utilizó el algoritmo real de IBVS, con alguna cierta modificación. La geometría del objeto a detectar, para el uso de IBVS, puede resultar un problema, ya que este algoritmo es más complejo que la primera implementación de prueba. En el primer código, no se tiene en cuenta la orientación del objeto, simplemente se busca el centroide de este y se mueve al robot en dicha dirección, manteniendo la orientación de la mano fija, mirando al suelo. Aquí nos encontramos con un problema, y es que IBVS, utilizando la matriz de interacción L_s , orienta también la cámara para alinear las esquinas actuales con las deseadas, pero el objeto es esférico, del mismo color, y por eso si el robot visualiza el objeto de lado, va a seguir viendo las 4 esquinas de la *bounding box*. Y es aquí donde viene el problema, al inclinar la cámara hacia un lado del objeto, realmente el robot no se está acercando al objeto, pero visualmente, las esquinas de este se acercan al centro de la imagen, a las esquinas deseadas. Además, no se puede simplemente anular la rotación, porque el controlador va a seguir intentando orientar al robot lateralmente.

Para solucionar esto, se propuso una solución en la detección de la pelota. Al detectar el objeto y encerrarlo en una caja, se obtiene un cuadrado en la imagen. Lo que se va a hacer es proyectarlo en el plano, corregirlo en orientación para que esté paralelo al plano, y reprojectarlo de vuelta en la imagen. De este modo, si el robot intenta girar lateralmente, el *bounding box* en vez de dirigirse hacia el centro de la imagen, se va a alejar y a hacerse más rectangular. De este modo, ya se puede proceder con el algoritmo de IBVS.

Se comienza creando los 4 puntos de las esquinas actuales y los de la meta, y estimando la profundidad mediante el lado del cuadrado generado. Una vez hecho esto, se calcula la matriz de interacción, utilizando las focales de la cámara utilizada, la Intel RealSense D455, quedando de la siguiente manera:

$$L_i = \begin{bmatrix} -\frac{f_x}{z} & 0 & \frac{u}{z} & \frac{u v}{f_x} & -\frac{f_x^2 + u^2}{f_x} & v \\ 0 & -\frac{f_y}{z} & \frac{v}{z} & \frac{f_y^2 + v^2}{f_y} & -\frac{u v}{f_y} & -u \end{bmatrix}$$

Para la construcción de la matriz de interacción se utilizan 3 submatrices 2x6, una por cada esquina. Al utilizar 3 esquinas, se obtiene una matriz cuadrada, 6x6, lo cual aporta más estabilidad a los cálculos, al poder calcular la inversa. Las variables u y v es el error en píxel entre la esquina actual y la deseada, z es la profundidad estimada y f_x y f_y son las focales de la cámara.

Al igual que en la anterior implementación, se utiliza un segundo controlador para mantener la orientación perpendicular al plano. Esto se hace ya que, debido al propio movimiento generado por el controlador visual,

se generaba una rotación en el *end effector* que introducía error, e incluso podía provocar que el objeto desapareciese de la escena. Por otro lado, también se implementó un controlador PD para lograr que el comportamiento generado fuese más exacto.

Por último, es importante destacar que el control visual genera las velocidades cartesianas deseadas en la cámara, por lo que se tiene que implementar cinemática inversa, en este caso con *Damping Least Squares* para mayor estabilidad, para obtener la velocidad articular deseada. Se intentó utilizar directamente la velocidad articular en el controlador, pero el comportamiento era erróneo, por lo que se optó por calcular la configuración articular deseada, conociendo la diferencia temporal. De este modo, se pudo controlar el robot correctamente.

5.5.2. Control visual en el robot real: AprilTags

Una vez el comportamiento ha sido implementado y demostrado en el simulador, ya se puede pasar a la siguiente fase: el robot real. Para controlar el robot real se ha decidido utilizar ViSP, ya que es una librería pensada para control visual y facilita mucho la implementación con el robot real, dado que utiliza la librería de *libfranka* para controlar directamente al robot. Además, esto es muy útil, ya que crea directamente un controlador en impedancia para el Franka Research 3 que utiliza como parámetro de control la velocidad generada por el control visual. Dicho esto, es necesario obtener los parámetros intrínsecos y extrínsecos de la cámara para esta parte, por lo que se utilizó el módulo de autocalibración, explicado en el Apartado 5.3.2.

La primera implementación se tratará de utilizar las librerías de las que dispone ViSP para detectar y obtener la pose de AprilTags (ver **Figura 5.6**). Este tipo de elementos son muy útiles en robótica, ya que simplemente detectándolos se puede conocer con bastante exactitud la pose del objeto y la distancia al mismo.

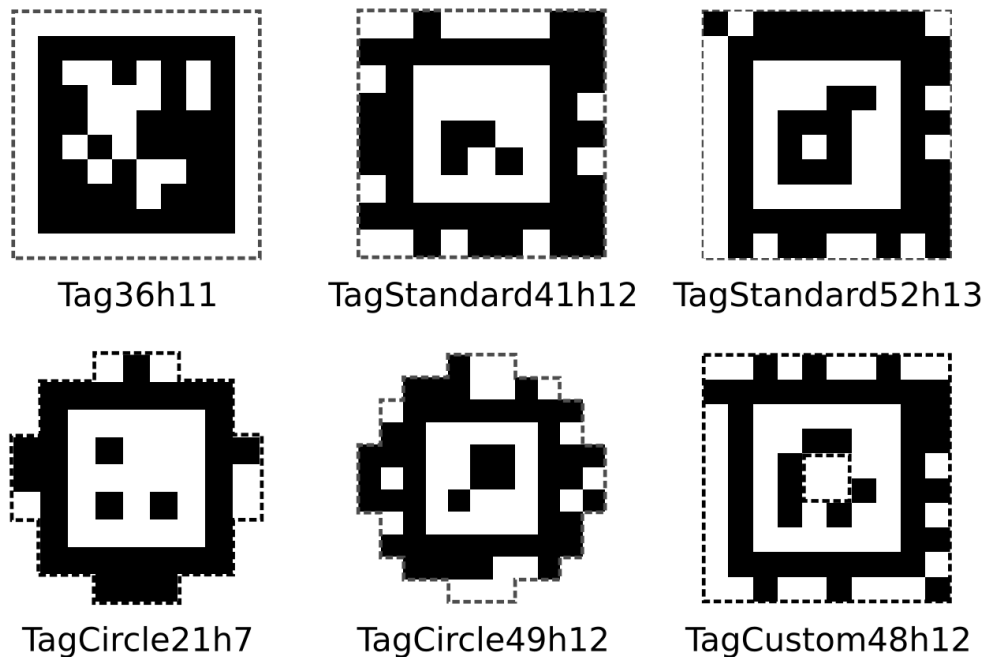


Figura 5.6: Diferentes tipos de AprilTags utilizados en robótica

Al utilizar ViSP no es necesaria la implementación de la matriz de interacción, ViSP se encarga de crearla y utilizarla. En este caso, lo que se tiene que hacer es obtener la matriz de transformación homogénea M_o^c , para obtener la transformación de las coordenadas del objeto a la cámara, y entonces se proyecta en la imagen para poder aplicar IBVS de manera precisa. La clase `vpDetectorAprilTag` ya devuelve automáticamente esta matriz, así que lo que hay que hacer es utilizarla para las proyecciones. De este modo tenemos la información tridimensional del objeto, es decir, la posición de las 4 esquinas, la rotación y la profundidad. Se utiliza la función `computeControlLaw()` para obtener las velocidades deseadas de la cámara, y se mueve el robot, respetando la frecuencia de 1kHz.

5.5.3. Control visual en robot real: Seguimiento de la pelota

Una vez se ha hecho el programa de prueba con los AprilTags se elabora la implementación para conseguir el objetivo real del proyecto, hacer que el robot realice el seguimiento de una pelota azul. Para esto se parte del mismo código utilizado para los AprilTags, sustituyendo las funciones de ViSP por las propias. Primero de todo, hay que conseguir detectar la pelota. Para ello, se implementó el mismo tipo de detección que en simulación, por RGB, pero el problema en la realidad viene por la iluminación. En simulación la iluminación es perfecta, y se detecta sin ningún tipo de problema la pelota, pero en la realidad, y con la cámara real, el azul no siempre se percibe como azul. Al variar la iluminancia de la escena, se puede aproximar más al color negro o al blanco, existen reflejos dentro de la pelota, e incluso en el borde de la mesa se refleja el color de la misma, apareciendo en color azul también. Por ende, se realizaron varias pruebas para afinar lo máximo posible esta detección, buscando que el *bounding box* se aproxime a un cuadrado y utilizando la mediana de un historial de 15 detecciones, para disminuir la oscilación de la caja.

En este caso, y al contrario que en simulación, no es necesario calcular explícitamente la profundidad. Simplemente se utiliza el radio de la pelota para calcular las esquinas meta, y eso es suficiente para el algoritmo de IBVS que implementa ViSP para alcanzar la profundidad deseada. A pesar de utilizar una cámara RGBD, con la que se puede saber la profundidad real al objeto, no se utiliza la componente de profundidad. Esto se debe a que el objetivo del proyecto es utilizar una cámara de eventos, no una cámara RGBD, por ende, utilizar para esta implementación la profundidad aportada por la cámara, sería “hacer trampas” para el futuro. Por otro lado, debido a cómo funciona el método para declarar las esquinas actuales, es necesario implementar la función `sortCornersForTarget()` para cerciorarse de que estén en el orden correcto. Una vez tenemos esto, se vuelve a utilizar la función `computeControlLaw()` para controlar al robot.

Tras realizar esta implementación, se decidió intentar mejorar la parte de detección de la pelota, ya que el control visual funcionaba perfectamente, pero la detección podía oscilar un poco y dependía de la iluminación de la escena para su correcto funcionamiento. Se propuso hacer un *fine-tuning* de la red neuronal de YOLOv4 [42] para detección de objetos, utilizando un *dataset* de la pelota que iba a tener que seguir en la realidad el robot. Para la creación de dicho *dataset* se utilizó la herramienta de MakeSense AI [43] para conseguir el *ground truth* de la detección, es decir, lo que se le va a pasar a la red neuronal como detección correcta. Se creó un *dataset* compuesto por 60 imágenes, 50 de entrenamiento y 10 de test, capturadas con la RealSense D405 montada en el robot. Se situó el efector final del robot en diferentes poses, y se utilizaron diferentes tipos de iluminación, para que el *dataset* fuese robusto ante todo tipo de condiciones lumínicas de la escena, para de este modo arreglar el problema presente en la anterior implementación.

Una vez se obtuvo la red neuronal optimizada para encontrar la pelota, se creó la clase `PelotaDetector`, para poder realizar la detección utilizando la librería de *OpenCV* para redes neuronales en C++. La clase utiliza la red para detectar la pelota dentro de la imagen de ViSP que recibe, y devuelve la *bounding box* generada, utilizando también el filtrado de la mediana para que sea más robusto a oscilaciones en la detección. En el código principal sólo hay que modificar la función `detector()` para que, en vez de llamar a la función creada en la implementación anterior, llame a esta nueva clase.

5.5.4. Control visual con cámara de eventos

En este apartado se explicará la implementación final para alcanzar el objetivo principal propuesto en este TFG, utilizar un control visual dinámico de un robot colaborativo con una cámara de eventos montada *eye-in-hand*. Este apartado, al converger 2 subproyectos separados, ha sido realizado en colaboración con Gabriel Barrigón Gómez [36], llevando él a cabo la parte de visión con la cámara de eventos.

El código está separado en 2 procesos, un servidor, que es el que utiliza la detección de la pelota con la cámara de eventos, y el proceso de ViSP para el control visual. Para comunicar ambos procesos se utilizó un protocolo TCP [44], una manera rápida y sencilla de comunicación eficaz. Para esta implementación se tuvo que hacer una modificación en el control visual, eliminar la profundidad, y hacer que el robot siga a la pelota sin variar la altura. Se ha hecho de este modo porque la detección de la pelota varía mucho, porque también depende de la cantidad de eventos generados, si la pelota o el robot se mueven rápido, hay más eventos y se puede detectar

mejor la pelota, si no, hay menos y se puede detectar parcialmente. Por eso mismo, no se puede implementar directamente el control en altura, ya que, al variar la detección, y estimar la profundidad a partir del área del *bounding box*, el robot oscilaría demasiado en el eje z.

Por otro lado, para hacer más sencillo el control, se propuso no utilizar las 4 esquinas que encierran al objeto, sino el centroide del mismo, para de este modo simplificar el seguimiento de la pelota. Realmente no se necesita utilizar para esta implementación las 4 esquinas, ya que la información que nos aporta es para la orientación, pero al tratarse de una esfera, no se necesita rectificar en orientación.

Con esta implementación final, se da por concluido el apartado de desarrollo de este TFG, habiendo logrado obtener resultados, no solo en el agente de Deep RL en la procura de mejorar el control en impedancia, sino que también en la tarea de control visual con una cámara de eventos.

6. RESULTADOS

En este apartado se hablará de los resultados obtenidos en el desarrollo de este Trabajo Fin de Grado. En primera instancia, se repasarán los resultados de las diferentes implementaciones, modificaciones e hipótesis planteadas en el agente de Deep RL para la obtención de las ganancias para el control en impedancia. Tras esto, se detallarán los resultados obtenidos en el control visual, observando los vídeos generados de las diferentes implementaciones.

6.1. Control en impedancia

Para el monitoreo de los resultados de la implementación del agente de Deep RL para la tarea de generar las ganancias necesarias para el control en impedancia, se utilizó la suite de visualización *TensorBoard* (de la librería de *TensorFlow*) [45]. Gracias a esta librería se puede configurar el entorno del agente para generar las gráficas del entrenamiento típicas (*actor loss*, *critic loss*, recompensa media, etc) y los parámetros deseados (promedio del *jerk*, torques generados, entropía, etc).

6.1.1. Modificaciones en el agente

En este apartado, se irán analizando los resultados de varias modificaciones implementadas en el agente, comenzando por una de las primeras, la normalización de la recompensa y el espacio de observación y acción. Esta modificación ha resultado de gran importancia para el entrenamiento, ya que ha reducido drásticamente la explosión de la función de pérdida de la red *critic*, encargada de evaluar la política generada por el agente.

En la **Figura 6.1** se puede observar que, al no haber normalizado los parámetros, el *critic loss* alcance valores muy altos, y se queda oscilando en este orden de magnitud. Esto provoca que la política generada por la red *actor* no esté correctamente evaluada, por lo que la red *actor* cree que está aprendiendo correctamente, cuando no es cierto. En la **Figura 6.2** se muestra como al normalizar estos parámetros del agente la función pérdida ahora no explota, y va descendiendo hasta estar oscilando en unos valores bajos. Esto es normal, ya que la red aún sigue intentando aprender qué es lo correcto y qué lo incorrecto, es decir, a evaluar correctamente la política.

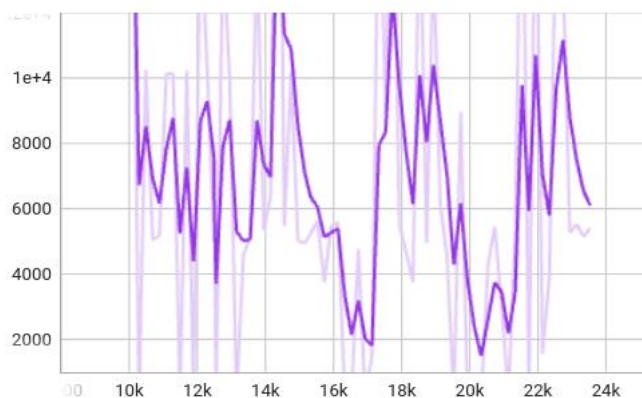


Figura 6.1: *Critic loss* antes de la normalización

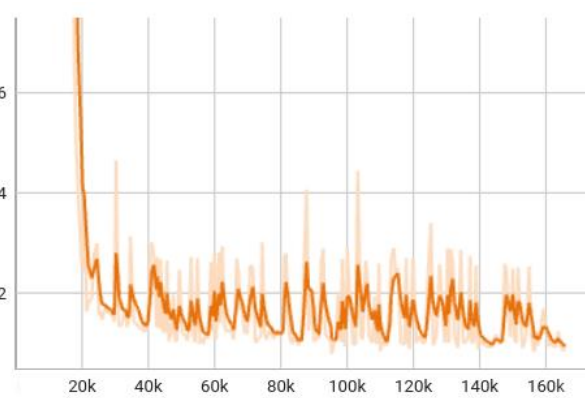


Figura 6.2: *Critic loss* después de la normalización

Por otra parte, un parámetro muy importante en el algoritmo SAC, que es justamente lo que introduce como novedad y su principal característica, es la entropía. La entropía ayuda al agente tanto a solucionar el problema exploración-explotación como a ser más robusto, una vez ha aprendido, ante el ruido y la variabilidad de las mediciones. Un problema presente en la implementación ha sido la explosión de la entropía (ver **Figura 6.3**), la cual provocaba que al caer a un valor cercano a 0 el agente sólo explotaba el conocimiento, sin explorar más; por ende, la recompensa media del episodio se estancaba, oscilando en torno a un valor (ver **Figura 6.4**).

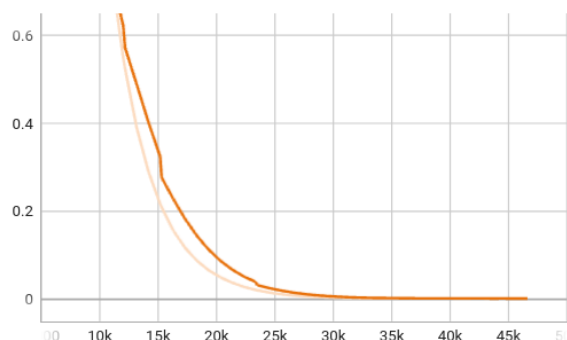


Figura 6.3: Explosión de la entropía: coeficiente de entropía

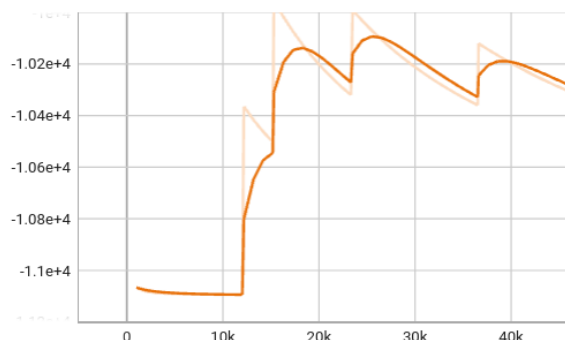


Figura 6.4: Explosión de la entropía: recompensa media del episodio

Si se estabiliza la entropía, haciendo que decaiga más lentamente y alcanzando un valor mínimo establecido, el agente es capaz de seguir explorando, y esto se observa viendo como la recompensa media del episodio ahora no se estanca, sino que fluctúa, buscando nuevas maneras de intentar maximizar la recompensa. Este comportamiento se observa en la **Figura 6.5** y la **Figura 6.6**.

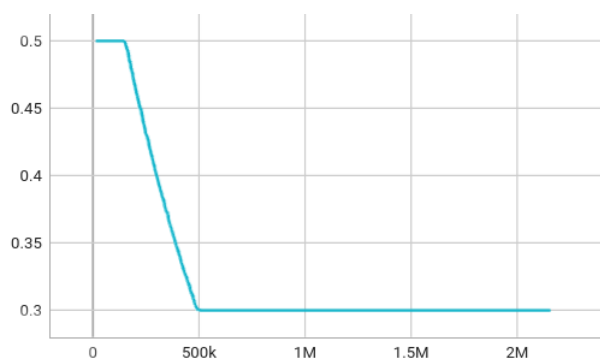


Figura 6.5: Estabilización de la entropía: coeficiente de entropía

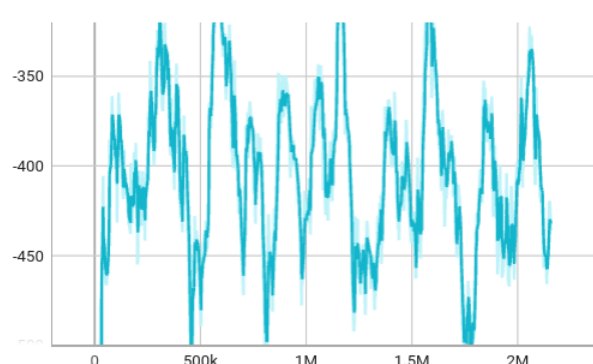


Figura 6.6: Estabilización de la entropía: recompensa media del episodio

Tras esto, se seguía observando que el agente no era capaz de aprender una política determinada, empezaba a aprender un comportamiento que aumentaba la recompensa media, pero para luego desaprenderlo, volviendo a reducir la recompensa. Esto se repite constantemente, obteniendo una recompensa media oscilatoria, como se observa en la última imagen (**Figura 6.6**).

Las últimas pruebas realizadas se corresponden a la implementación de la generación de ganancias siguiendo un perfil trapezoidal. Se intentaba incitar al agente a que aprendiese a generar las ganancias siguiendo este tipo de función, con lo cual se llegó a la siguiente política. En la **Figura 6.7** se puede observar la longitud media del episodio, es decir, cuántos *steps* le hacía falta de media a un episodio para llegar a la meta. La **Figura 6.8** muestra la recompensa media del episodio, lo que nos indica cómo está aprendiendo el agente. La recompensa demuestra una clara subida, oscilando entre los valores de 0 a -100, lo que indica que el agente ha aprendido una política que aumenta la recompensa recibida; pero si se observa la longitud media del episodio, se logra comprender qué es lo que está aprendiendo el agente. Básicamente, el agente ha aprendido a reducir el tiempo que tarda en alcanzar la meta, pero nada más.

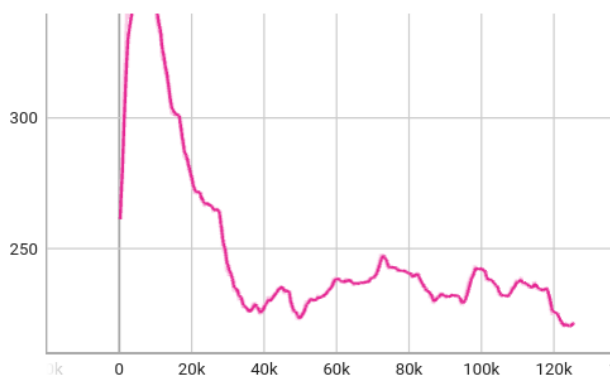


Figura 6.7: Longitud media del episodio

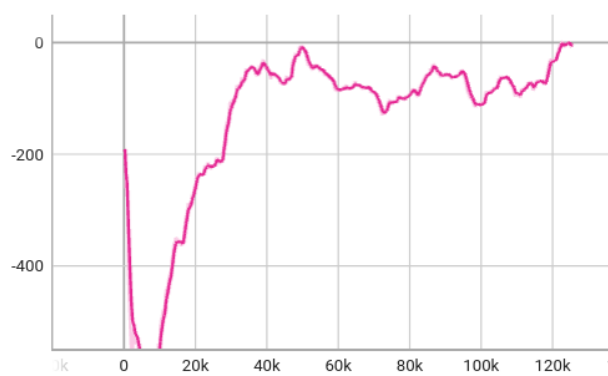


Figura 6.8: Recompensa media del episodio

El *jerk* generado sigue siendo el mismo, y como se aprecia en la **Figura 6.9**, no ha logrado aprender a generar las ganancias siguiendo un perfil trapezoidal. Esta última gráfica muestra la evolución del parámetro *count_D*, que expresa el número de articulaciones, de media, que ha incumplido el perfil trapezoidal.

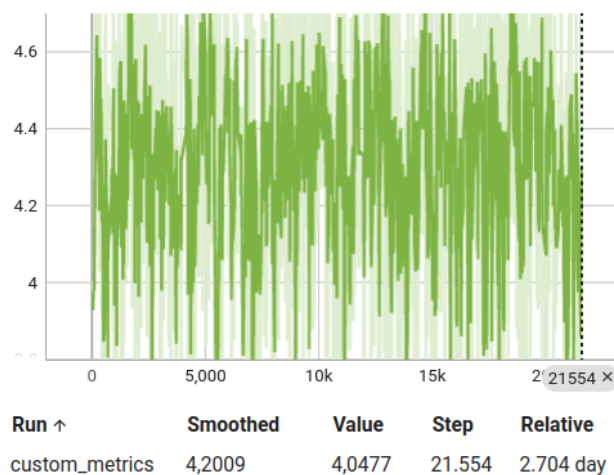


Figura 6.9: Evolución del parámetro *count_D*

6.1.2. Variantes en la implementación

En este apartado se analizarán los resultados de las propuestas sobre nuevos enfoques de la implementación que se han llevado a cabo en el desarrollo. Para analizar la viabilidad de estas implementaciones se creó un comportamiento donde el robot partía de la configuración articular de reposo (ver **Figura 5.4**) y se dirigía a otra pose, extendiendo el brazo hacia arriba y hacia un lado, para luego volver a la pose *home*. Durante esta trayectoria, se guardaron los datos de la distancia y rotación a la meta, la velocidad articular, el *jerk* y los torques generados. Es importante tener en cuenta que el tiempo medido no es el tiempo de simulación, sino que el tiempo real. Al haber hecho las modificaciones en la frecuencia para que opere a 1kHz, la relación entre el tiempo de simulación y el tiempo real es de 1:30, es decir, que 1s en simulación son 30s de tiempo real.

Primero, se observarán los resultados para el control en impedancia normal, con unas ganancias fijas. Para estas pruebas se han utilizado las mismas ganancias para todas las articulaciones. Con unas ganancias altas, de $K = 600$ y $D = 60$, se observa que el robot es capaz de llegar muy rápido a la pose meta (ver **Figura 6.10**) pero genera un *jerk* muy grande y no admisible por el robot (ver **Figura 6.11**), al realizar cambios en la aceleración muy bruscos. Además, es importante fijarse en un hecho importante, estos cambios bruscos en la aceleración provocan unos picos muy altos, justamente cuando inicia (o cambia) el movimiento. Esto sucede porque el controlador intenta partir del reposo a una velocidad muy grande de forma instantánea, y es por eso por lo que se ven 2 grandes picos, que se corresponden a cuando inicia el movimiento hacia la siguiente pose. Si se

observa la evolución de las velocidades articulares en la **Figura 6.12**, se apreciará que esto es cierto, y que se producen unos saltos donde se inicia el movimiento.

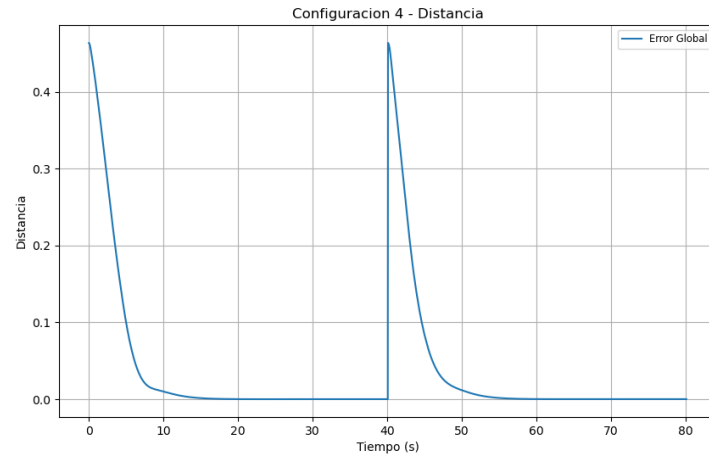


Figura 6.10: Evolución de la distancia a la meta para $K = 600$

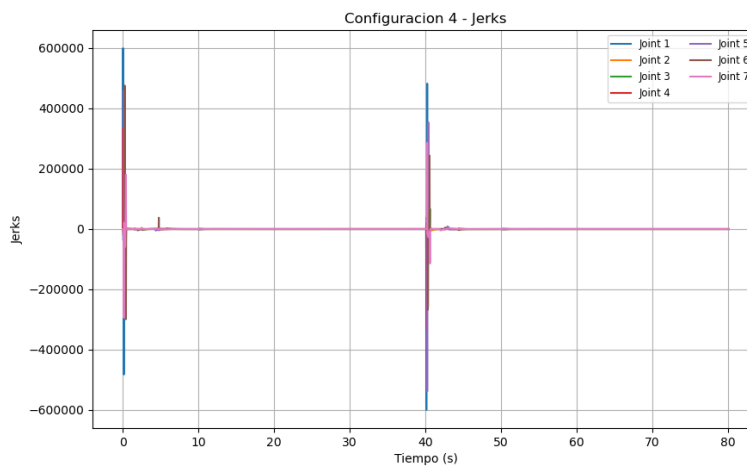


Figura 6.11: Evolución del *jerk* para $K = 600$

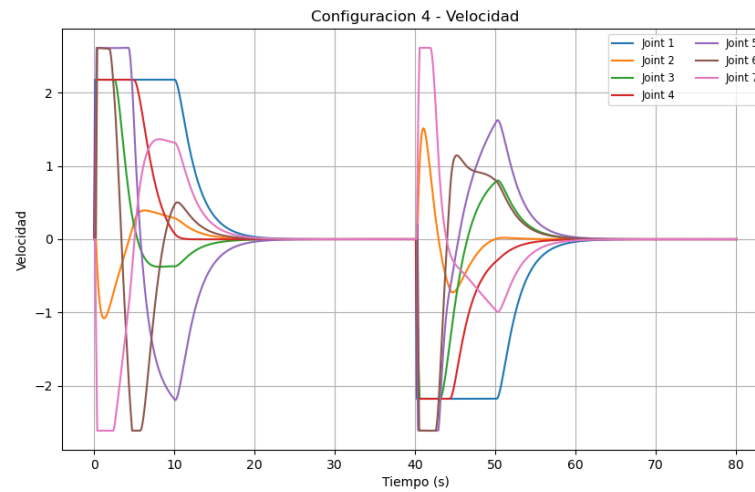


Figura 6.12: Evolución velocidades articulares para $K = 600$

Si se utilizan unas ganancias más pequeñas, con una $K = 300$ y $D = 30$, ahora el robot tarda un poco más en llegar a la meta (ver **Figura 6.13**), aun así, siendo muy rápido, pero los *jerks*, aunque no admisibles para el robot, han bajado (ver **Figura 6.14**). Con esto, se puede ver una tendencia a que, al reducir las ganancias, el *jerk* generado será menor, pero a costa de la velocidad del brazo. Esta tendencia tiene un sentido teórico, ya que lo que se está haciendo al reducir la ganancia K es hacer el robot más “blando”, por lo que le va a costar un poco más de trabajo llegar a la meta, pero al ser menos rígido, los saltos en la aceleración, que son los que provocan el *jerk*, se van a reducir.

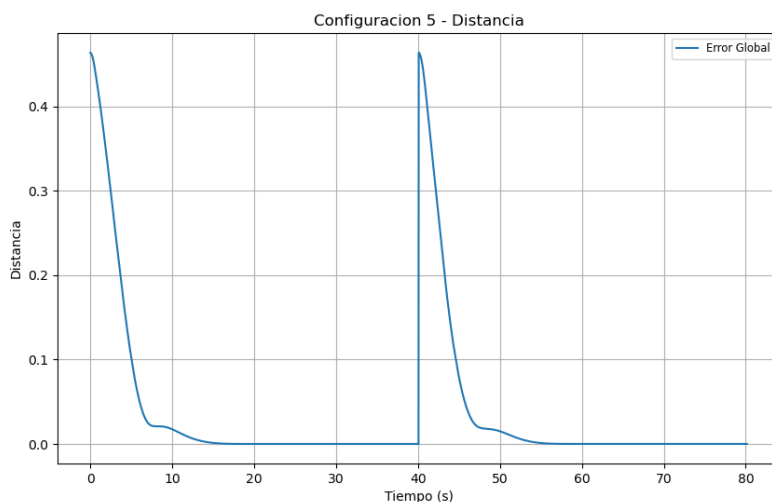


Figura 6.13: Evolución de la distancia a la meta para $K = 300$

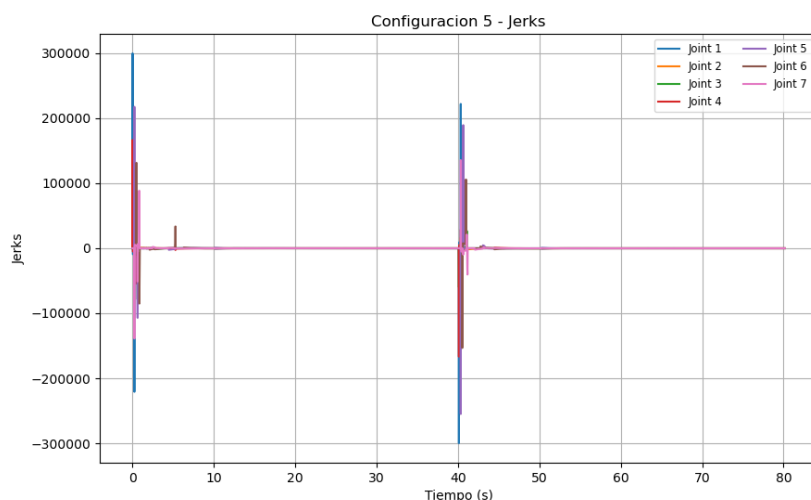


Figura 6.14: Evolución del *jerk* para $K = 300$

Sabiendo esto, lo que se planteó fue llevar a cabo un controlador que tuviese en cuenta esta información, para intentar que la velocidad no produjese saltos, sino que llevase una curva más lineal y suave. La primera implementación que se utilizó fue la generación de ganancias mediante un perfil trapezoidal (Apartado 5.4.7), siguiendo la teoría de que, con ganancias muy bajas, se podrá reducir estos saltos en la velocidad. En la **Figura 6.15** se puede observar que ahora el robot tarda más en llegar a la meta, pero si calculamos el tiempo real dividiendo entre 30, se obtiene que el robot tarda 1.67s aproximadamente en alcanzar la pose meta, que está a cerca de 0.5m de distancia. Para un robot manipulador, este tiempo de pasar de una pose en reposo y alcanzar otra, también en reposo, es bastante buena y rápida.

Lo más interesante es observar la evolución de los *jerks* siguiendo este tipo de perfil, en la **Figura 6.16**. Ahora, el pico más alto está en $-1500 \frac{rad}{s^3}$, cuando el robot puede soportar hasta $5000 \frac{rad}{s^3}$ en cada articulación, como se puede leer en la **Tabla 4.2**. También se observa que, aunque los picos son mucho menores (antes eran del

orden de $300.000 \frac{rad}{s^3}$), ahora hay más de ellos. Esto sucede al ir variando las ganancias, ya que esa variación también aporta saltos en las velocidades al sistema. Si ahora se observa la evolución de las velocidades articulares, en la **Figura 6.17**, se puede ver que ahora siguen una curva lineal y suave, pero hay un dato muy importante que se puede apreciar, la evolución de la articulación 7. En la gráfica de la velocidad, se observa que en el instante $t = 6s$, más o menos, esta articulación empieza a moverse, cuando el sistema ya está aplicando unas ganancias más altas. Si ahora observamos la evolución de los *jerks*, los picos más altos los produce esta articulación, y esto sucede porque para estas pruebas se está utilizando una ganancia global para todas las articulaciones. Si desacoplamos las ganancias y las hacemos locales a cada articulación, el *jerk* generado también será menor.

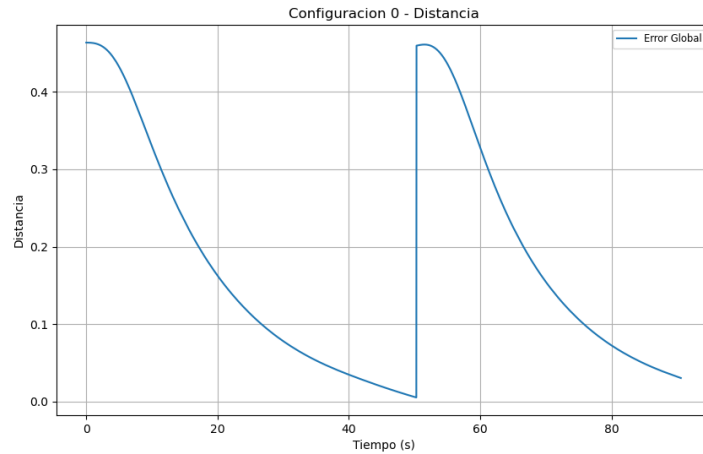


Figura 6.15: Evolución de la distancia a la meta con ganancias generadas siguiendo un perfil trapezoidal

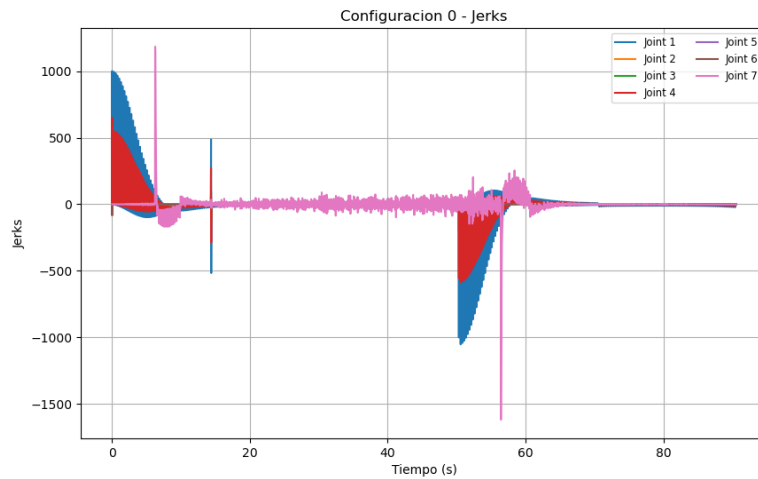


Figura 6.16: Evolución del *jerk* con ganancias generadas siguiendo un perfil trapezoidal

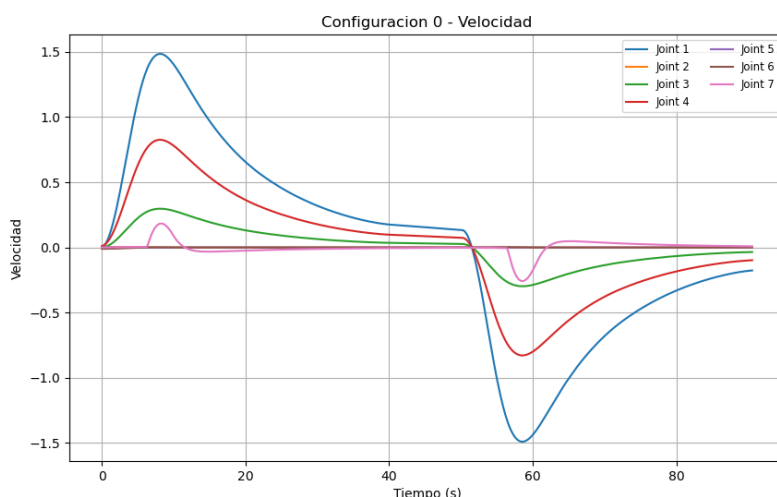


Figura 6.17: Evolución de las velocidades articulares con ganancias generadas siguiendo un perfil trapezoidal

La siguiente implementación que se llevó a cabo es el control basado en aceleración, explicado en el Apartado 5.4.8. Esta implementación, basada en la arquitectura del control en impedancia con el robot real, pretende seguir una aceleración constante en todo el movimiento, para de este modo reducir drásticamente el *jerk* generado. Esencialmente, se desarrolla un perfil trapezoidal para la velocidad, donde la pendiente es la aceleración constante utilizada, pero no necesariamente tiene que alcanzar el estado estacionario del perfil trapezoidal (donde la velocidad se vuelve constante), ya que eso sucede si llega a la velocidad máxima. Además, se utilizaron unas ganancias K y D fijas, con los mismos valores que la primera prueba, con $K = 600$ y $D = 60$ y luego $K = 300$ y $D = 30$.

En la **Figura 6.18** se aprecia que, con la aceleración utilizada, se tarda más en llegar a la meta, 2.67s, pero esta implementación es simplemente de prueba, y se puede hacer más veloz aumentando la aceleración constante. Lo importante viene en la **Figura 6.19**, donde se observa que ahora el *jerk* generado es muy pequeño, con unos valores que oscilan generalmente entre $-50 \frac{\text{rad}}{\text{s}^3}$ y $50 \frac{\text{rad}}{\text{s}^3}$, salvo un pico inicial de $-250 \frac{\text{rad}}{\text{s}^3}$ (aun así muy pequeño). Ahora, en la **Figura 6.20**, se observan las velocidades articulares generadas, que ahora presentan una curva muy suave. Por otro lado, se puede observar que, en este caso, en el *jerk* sólo ha habido un pico, cuando normalmente había 2 (uno por cada traslación). Esto se debe a que, si se observan las velocidades generadas, al volver por la misma trayectoria, pero en sentido contrario, las velocidades generadas forman una recta, por lo que es el tramo con menor *jerk* generado.

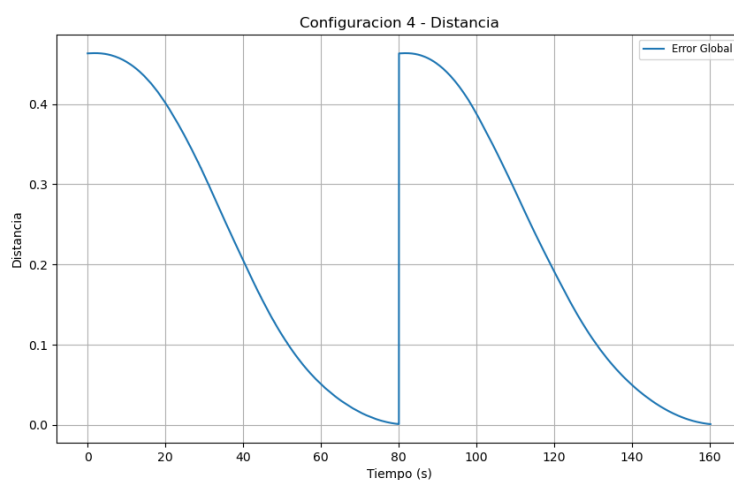


Figura 6.18: Evolución de la distancia a la meta con aceleración constante y $K = 600$

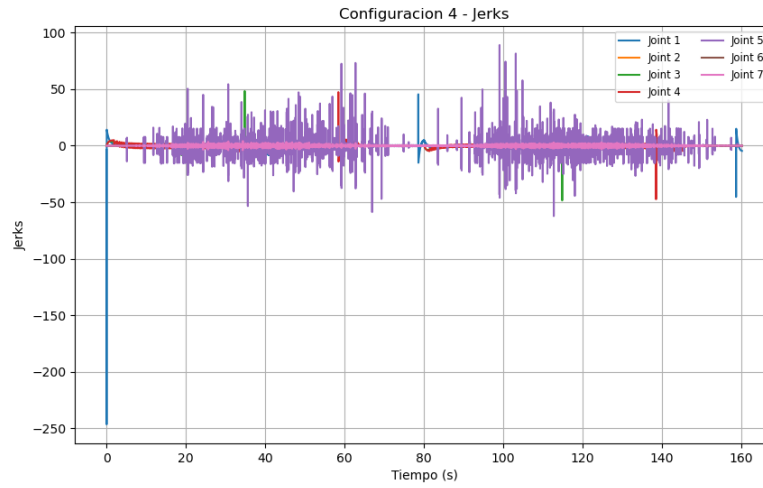


Figura 6.19: Evolución del *jerk* con aceleración constante y $K = 600$

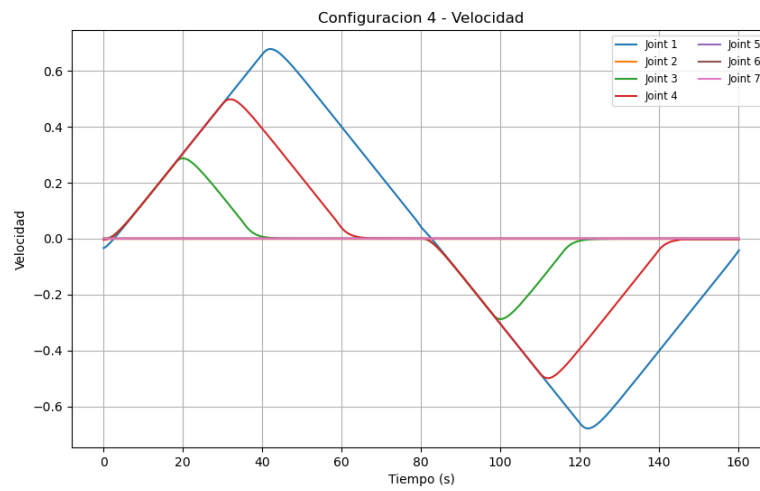


Figura 6.20: Evolución de las velocidades articulares con aceleración constante y $K = 600$

Pero hay un hecho muy importante sobre esta implementación, y es que, al reducir las ganancias a la mitad, no se observa ningún tipo de cambio en la salida del sistema. Las gráficas generadas de *jerk* y velocidades articulares prácticamente coinciden, utilizando valores distintos de las ganancias (ver **Figura 6.21** y **Figura 6.22**)

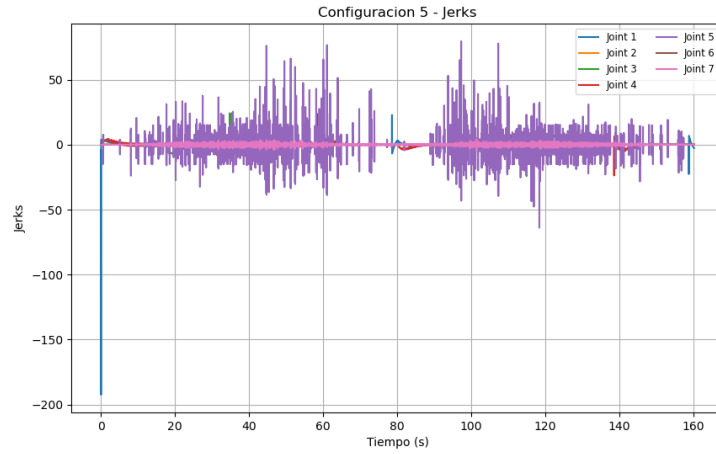


Figura 6.21: Evolución del jerk con aceleración constante y $K = 300$

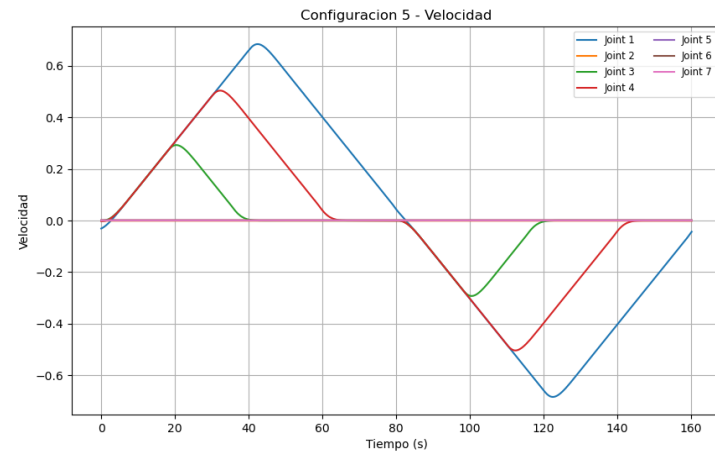


Figura 6.22: Evolución de las velocidades articulares con aceleración constante y $K = 300$

6.2. Control visual

Para el apartado de control visual se utilizarán los vídeos grabados en el entorno de trabajo, para tener perspectiva de todo el comportamiento del robot. Se ha utilizado una cámara con una perspectiva cenital, otra con una perspectiva general del entorno, y finalmente una captura de pantalla, donde se observa la vista de la cámara RealSense D405 montada en el robot, con todos los datos del seguimiento de la pelota. A pesar de poder funcionar sin errores a mayores velocidades, por motivos de seguridad del robot en las pruebas se ha decidido disminuir la velocidad operacional del robot, a falta de un controlador que mantenga al robot dentro de un espacio de trabajo delimitado en las pruebas.

6.2.1. Simulación

Para la parte de la simulación se han utilizado 2 vídeos de la implementación, uno de la perspectiva general del simulador (*sim_visual_servoing_camara.mp4*) y otro de la vista de la cámara montada en el robot con el *bounding box* actual y el deseado (*sim_visual_servoing_pantalla.mp4*). Para la prueba, la pelota parte de una posición a la izquierda del robot, y se aprecia como el *end effector* se sitúa justo encima de ella, manteniendo la posición correctamente. Al pasar un determinado tiempo, la pelota se mueve hacia la derecha del robot, y el robot es capaz de seguir el movimiento sin ningún problema, volviendo a situarse encima del objeto. El código utilizado para la prueba es el que utiliza IBVS.

6.2.2. Robot real: AprilTags

La siguiente implementación es el seguimiento de un AprilTag, en los vídeos *apriltag_cenital.mp4*, *apriltag_escena.mp4* y *apriltag_pantalla.mp4*. En estos vídeos se puede observar que el comportamiento del seguimiento con el AprilTag es totalmente perfecto. Al obtener una información precisa de la pose del objeto, el robot es capaz de situarse encima de él, corregir la orientación perfectamente y mantener la distancia deseada con este. Se aprecia como al levantar el AprilTag el robot sube el efector final para mantener en todo momento la distancia. Además, al tratarse de una figura con un patrón de cuadrados negros y blancos, es invariante a la iluminación de la escena.

6.2.3. Robot real: Pelota con cámara RGBD

En el caso de la primera implementación del seguimiento de la pelota, con el detector por RGB, los resultados de las 2 pruebas están disponibles en los vídeos *pelota_rgb_cenital.mp4*, *pelota_rgb_escena.mp4* y *pelota_rgb_pantalla.mp4* (1 y 2). Con la iluminación actual el seguimiento es dinámico y suave, sin ninguna oscilación que perjudique al movimiento. Además, la estimación de la profundidad de la pelota funciona bastante bien, manteniendo la profundidad de manera suave, sin perturbaciones. Por otro lado, se puede observar que la detección, gracias a utilizar el filtro de la mediana de un cierto número de detecciones, es bastante fluida, sin oscilaciones aparentes en el tamaño de la misma. Este filtro provoca un cierto *delay* en la detección, pero al ser bastante ligero, no afecta en los resultados.

Para la implementación utilizando YOLOv4 primero hay que observar los resultados del entrenamiento. Como solo hay una clase, lo que más nos interesa en estos resultados es conocer la media de IoU obtenido en el entrenamiento. El IoU (*Intersection over Union*) es una medida para conocer cómo de precisa es la colocación de la *bounding box* en el objeto a detectar. Se calcula por el porcentaje de área de la caja generada que coincide con el *Ground Truth*. En este caso, el IoU obtenido es de 0.95, es decir, que el 95% del área de la caja generada coincide con lo que se usa como caja verdadera. Además, como se puede apreciar en la **Figura 6.23**, la detección en las imágenes de test es excelente, y corrobora el IoU obtenido.

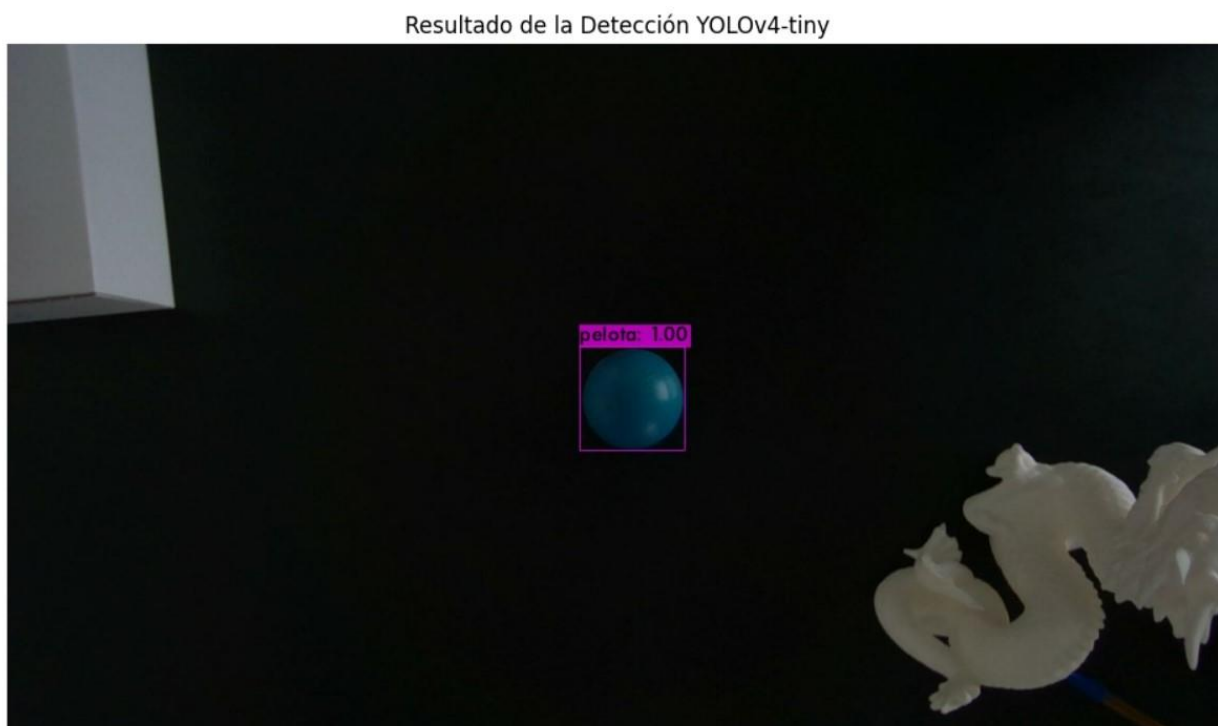


Figura 6.23: Resultados del entrenamiento de YOLOv4

Los resultados de la implementación del control visual se reflejan en los vídeos *pelota_yolo_cenital.mp4*, *pelota_yolo_escena.mp4* y *pelota_yolo_pantalla.mp4*. Con la detección mediante YOLOv4 los resultados son bastante diferentes a los anteriormente expuestos. Se aprecian ciertas vibraciones en el robot, a medida que se

va acercando a la pelota, y, además, una vez se sitúa encima, le cuesta disminuir el error hasta el umbral establecido. Esto se debe a que, si se observa la *bounding box* generada en el vídeo, esta varía mucho, generando formas más rectangulares durante varios *frames* seguidos. A pesar de aplicar el filtro de la mediana, en este caso de 30 detecciones en vez de 15, se siguen observando ciertas oscilaciones en el control, a pesar de que los resultados aparentes del entrenamiento de la red sean perfectos. La hipótesis principal de la causa de estos resultados es el *motion blur*. Cuando la pelota y el robot están estáticos, la red detecta correctamente la pelota (véase **Figura 6.23**), pero cuando el robot y/o la pelota están en movimiento, en la imagen se genera *motion blur*, es decir, un desenfoque en la imagen provocado por el propio movimiento de la cámara o de los objetos en la imagen. Además, el *dataset* utilizado es muy pequeño, de tan solo 50 imágenes para el entrenamiento, por lo que para solucionar este problema y lograr que YOLOv4 mejore considerablemente la detección de la pelota, habría que generar un conjunto de imágenes mucho más amplio, y con imágenes capturadas mientras la pelota se está moviendo, el robot se está moviendo y ambos, para intentar reducir las consecuencias del *motion blur*.

6.2.4. Robot real: Pelota con cámara de eventos

Esta es la implementación final de este Trabajo Fin de Grado, donde se reflejan los resultados obtenidos del objetivo principal del mismo: poder utilizar una cámara de eventos para una tarea de control visual por impedancia. Para el análisis de los resultados hay que centrarse en la parte del control visual, ya que es la que concierne a este TFG. En esta prueba, al no utilizar la cámara D405 para el control, se utilizó para grabar en vídeo las pruebas, por lo que en este caso hay 4 perspectivas diferentes registradas en los vídeos *pelota_eventos_cenital.mp4*, *pelota_eventos_frente.mp4*, *pelota_eventos_escena.mp4* y *pelota_eventos_pantalla.mp4*. En estos vídeos se pueden observar varias cosas. Primero de todo, al utilizar una cámara de eventos eliminamos la problemática del *motion blur*, debido al propio funcionamiento de este tipo de sensores neuromórficos. Además, también es robusto ante diferentes tipos de iluminación, ya que percibe cambios en la iluminancia de los píxeles y, además, tiene un amplio rango dinámico. La condición lumínica que sí puede interferir en el rendimiento es la oscilación de la fuente de luz. Es decir, si en vez de ser una fuente de luz fija (como un led) es una fuente variable (lámparas fluorescentes), se producen cambios en la iluminancia de la escena. Aún así, son unas condiciones de funcionamiento más robustas que las presentadas anteriormente.

Sin embargo, se observa como el robot puede oscilar un poco alrededor de la pelota, y esto es debido a que la detección puede variar bastante, según la velocidad de la pelota o el robot. No obstante, en estos vídeos se demuestra que el robot es capaz de seguir al objeto deseado correctamente.

7. CONCLUSIONES

Este es el último apartado de este Trabajo Fin de Grado, en el que se discutirá el éxito del mismo, en base a los resultados analizados y los objetivos propuestos y alcanzados, al igual que se definirá el trabajo a futuro del proyecto.

7.1. Control en impedancia

El primer objetivo del proyecto consistía en implementar el control en impedancia para el robot real, lo cual se ha logrado satisfactoriamente. El segundo objetivo, y el más ambicioso, consistía en elaborar un aprendizaje por refuerzo para obtener las ganancias necesarias para el control en impedancia, de manera dinámica y optimizada para la tarea y el movimiento generado por el robot. La idea original era, simplemente, implementar un algoritmo de Deep RL para que un agente fuese capaz de encontrar las ganancias óptimas en cada momento, pero sin una metodología definida. Con las modificaciones presentadas el agente no era capaz de aprender el comportamiento deseado, por lo que se estuvo estudiando la viabilidad del aprendizaje, para encontrar donde residía el error. Se ha planteado que la salida deseada era demasiado compleja para el algoritmo, al tratarse de generar los 14 valores distintos correctos para el movimiento deseado, por lo que se propone un nuevo método, basado en aprendizaje por curriculum. En este caso, no se podría utilizar SB3, ya que habría que alterar manualmente la salida de la red *actor*, haciendo que sólo tenga 2 neuronas de salida, una para K y otra para D . De este modo, en un principio se aprendería a generar las mismas ganancias para todas las articulaciones, se guardarían los pesos de la red (que en este caso debería de haber aprendido a generar correctamente las ganancias), y se cambiaría la salida por 14 neuronas, copiando los pesos de la neurona de la rigidez a las 7 primeras y los pesos de la neurona de amortiguamiento a las 7 siguientes, para que aprenda a optimizar la salida para cada articulación.

Por otro lado, en este TFG se han demostrado 2 implementaciones diferentes que, al menos en la teoría, funcionan adecuadamente. La generación de ganancias siguiendo un perfil trapezoidal ha demostrado que reduce considerablemente el *jerk* generado por el robot, y ha puesto en el punto de mira del control en impedancia la importancia de conseguir una aceleración suave y lineal. Por otro lado, la implementación del control basado en aceleración también ha demostrado reducir estas vibraciones, incluso más que el punto anterior; pero, sin embargo, la modificación de las ganancias K y D en esta estrategia de control carece de sentido, ya que no varían prácticamente la salida.

Por estos motivos, este Trabajo Fin de Grado propone como trabajo a futuro del proyecto estudiar esta nueva implementación de un *curriculum learning* más avanzado, y con la vía de utilizar estos 2 métodos que sí han dado buenos resultados.

7.2. Control visual

En este apartado se estudian los resultados sobre el objetivo final del proyecto, que era desarrollar un control visual con la información aportada de la cámara de eventos para el seguimiento de una pelota. Para las conclusiones de esta parte, es importante discernir entre las implementaciones con la cámara RGBD RealSense D405 y la implementación final con la cámara de eventos EVK4.

Por un lado, con la cámara RGBD queda demostrado, gracias a las pruebas realizadas con el AprilTag y la pelota con la detección mediante RGB, que el controlador funciona satisfactoriamente. Los únicos problemas percibidos han sido por culpa de la detección del objeto, y no por el controlador. Como trabajo a futuro del control visual mediante la cámara D405 se proponen 2 mejoras importantes:

- **Detección robusta:** La detección mediante YOLOv4 se presenta como la propuesta más favorable para aumentar la robustez de la implementación ante diferentes tipos de cambios en el entorno. Por ende, se pretende que en el transcurso futuro del proyecto se cree un conjunto de imágenes de mucha mayor riqueza y diversidad para solucionar los problemas presentados en la implementación.

- **Implementación de *tracker*:** Para lograr que el control visual sea más robusto ante posibles errores en la detección o que, simplemente, el objeto a seguir salga de la escena, se propone utilizar un *tracker* para poder estimar la pose del objeto aún cuando este deja de ser detectado. Concretamente, se propone utilizar MegaPose6D [46], un módulo dentro de ViSP que, no solo estima la posición del objeto, sino que devuelve la pose 6D, incluyendo la orientación, gracias a utilizar un modelo CAD del objeto. Esto permite no solo detectar la pelota utilizada, sino cualquier tipo de objeto.

Por otro lado, está la implementación final del proyecto, el control visual utilizando la información aportada por la cámara de eventos. En los vídeos aportados se demuestra que, con la detección actual de la pelota con la cámara EVK4, el control visual es capaz de situar el efector final del robot encima del objeto en movimiento. Por ende, los mayores obstáculos en el correcto seguimiento de la pelota con este tipo de sensor perceptivo, es la propia detección del objeto.

Los 2 puntos importantes por mejorar serían evitar la oscilación en torno a la pelota en el plano XY y, por otro lado, aportar un correcto cálculo de la profundidad de la pelota. Como solución a estos puntos se proponen 2 implementaciones:

- **Utilizar cámara RGBD:** Como con la detección actual no es posible estimar correctamente la profundidad a la pelota, se propone utilizar la cámara D405 en un lado del entorno. Al calibrarla en una posición fija y obtener los parámetros extrínsecos de la misma, se puede obtener la pose, con respecto al robot, de la cámara. Al utilizar la profundidad al objeto aportada por la D405, se puede situar en el espacio de la escena a la pelota. Como también se conoce el sistema de referencia del efector final, al pertenecer a la cadena cinemática del robot, se puede cerrar el ciclo y obtener la profundidad al objeto.
- **Filtrado de la detección:** Para reducir la oscilación del efector final en el plano XY, se propone como trabajo a futuro la implementación de un filtro para reducir las oscilaciones de la propia *bounding box* del objeto.

7.3. Reflexión final

Como conclusión de este Trabajo de Fin de Grado, cabe destacar que se han alcanzado satisfactoriamente los objetivos planteados, validando la viabilidad de la implementación propuesta y estableciendo una base robusta para futuras líneas de investigación.

Asimismo, dada la relevancia de los resultados obtenidos y la novedad de la aproximación utilizada, se está trabajando en la redacción de un artículo científico derivado de este proyecto con el propósito de ser presentado en la *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* [47], uno de los foros más prestigiosos en el ámbito de la robótica.

ANEXO A: LINKS PARA VÍDEOS Y CÓDIGOS

Link para los vídeos: [videos_TFG_Adrian_Fernandez_Casas](#)

Link para los códigos: https://github.com/Adrofercas/TFG_Adrian_Fernandez_Casas

REFERENCIAS

- [1] HOGAN, Neville. Impedance Control: An Approach to Manipulation: Part I—Theory. *Journal of Dynamic Systems, Measurement, and Control*. Marzo 1985, **107**(1), 1-7. ISSN 0022-0434.
- [2] INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. *Robots and robotic devices — Collaborative robots*. ISO/TS 15066:2016. Geneva: ISO, 2016.
- [3] SPONG, Mark W., HUTCHINSON, Seth y VIDYASAGAR, M. *Robot Modeling and Control*. Hoboken: John Wiley & Sons, 2006. ISBN 978-0-471-64990-8.
- [4] INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. *Robots and robotic devices — Vocabulary*. ISO 8373:2012. Geneva: ISO, 2012.
- [5] CHAUMETTE, François. Visual Servoing. En: DOMBRE, Etienne y KHALIL, Wisama (eds.). *Robot Manipulators: Modeling, Performance Analysis and Control*. London: ISTE Ltd; Hoboken: John Wiley & Sons, 2007, pp. 279-336. ISBN 978-1-905209-10-1.
- [6] GALLEGO, Guillermo, DELBRUCK, Tobi, ORCHARD, Garrick, et al. Event-based Vision: A Survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*. Enero 2022, **44**(1), 154-180. ISSN 0162-8828.
- [7] PROPHESSEE y SONY. *Prophesee and Sony Develop a Stacked Event-Based Vision Sensor with the Industry's Smallest Pixels and Highest HDR Performance* [en línea]. 19 de febrero de 2020 [consulta: 13 de enero de 2026]. Disponible en: <https://www.prophesee.ai/2020/02/19/prophesee-sony-stacked-event-based-vision-sensor/>
- [8] SUTTON, Richard S. y BARTO, Andrew G. *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: The MIT Press, 2018. ISBN 978-0-262-03924-6.
- [9] KOBER, Jens, BAGNELL, J. Andrew y PETERS, Jan. Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*. Septiembre 2013, **32**(11), 1238-1274. ISSN 0278-3649.
- [10] IGLESIAS RODRÍGUEZ, Roberto. *Robótica Adaptativa* [Material de clase]. Grado en Robótica. Santiago de Compostela: Universidad de Santiago de Compostela, 6 de febrero de 2025.
- [11] MNIH, Volodymyr, KAVUKCUOGLU, Koray, SILVER, David, et al. Human-level control through deep reinforcement learning. *Nature*. Febrero 2015, **518**(7540), 529-533. ISSN 0028-0836.
- [12] MNIH, Volodymyr, BADIA, Adria P., MIRZA, Mehdi, et al. Asynchronous Methods for Deep Reinforcement Learning. En: *Proceedings of the 33rd International Conference on Machine Learning (ICML)*. New York: PMLR, 2016, vol. 48, pp. 1928-1937.
- [13] LILLICRAP, Timothy P., HUNT, Jonathan J., PRITZEL, Alexander, et al. Continuous control with deep reinforcement learning. En: *4th International Conference on Learning Representations (ICLR)*. San Juan, Puerto Rico, mayo 2016.
- [14] ALBU-SCHÄFFER, Alin, et al. Cartesian impedance control of redundant robots: Recent results with the DLR-lightweight-robots. En: *IEEE International Conference on Robotics and Automation (ICRA)*. Roma: IEEE, 2007, pp. 3704-3709.
- [15] SERAJI, Homayoun y COLBAUGH, Richard. Force tracking in impedance control. *The International Journal of Robotics Research*. 1997, **16**(1), 97-117.
- [16] LOVE, L.J. y BOOK, W.J. Environment estimation for enhanced impedance control. En: *Proceedings of the 1995 IEEE International Conference on Robotics and Automation*. Nagoya: IEEE, 1995, vol. 2, pp. 1854-1859.
- [17] BUCHLI, Jonas, STULP, Freek, THEODOROU, Evangelos y SCHAAL, Stefan. Learning variable impedance control. *The International Journal of Robotics Research*. Junio 2011, **30**(7), 820-833. ISSN 0278-3649.

- [18] LUNIA, Akshit. *Deep Reinforcement Learning of Variable Impedance Control for Object-Picking Tasks*. Tesis de Máster. Delft: Delft University of Technology, 2022. Disponible en: https://open.clemson.edu/cgi/viewcontent.cgi?article=5282&context=all_theses
- [19] FUJIMOTO, Scott, VAN HOOFF, Herke y MEGER, David. Addressing Function Approximation Error in Actor-Critic Methods. En: *Proceedings of the 35th International Conference on Machine Learning (ICML)*. Estocolmo: PMLR, 2018, vol. 80, pp. 1587-1596.
- [20] CHAUMETTE, François y HUTCHINSON, Seth. Visual servo control. I. Basic approaches. *IEEE Robotics & Automation Magazine*. Diciembre 2006, **13**(4), 82-90.
- [21] MALIS, Ezio, CHAUMETTE, François y BOUDIRIGUE, S. 2-1/2-D visual servoing. *IEEE Transactions on Robotics and Automation*. Abril 1999, **15**(2), 238-252. ISSN 1042-296X.
- [22] LI, Yi-Rong, LIEN, Wei-Yuan, HUANG, Zhi-Hong y CHEN, Chun-Ta. Hybrid Visual Servo Control of a Robotic Manipulator for Cherry Tomato Harvesting. *Actuators* [en línea]. Junio 2023, **12**(6), 253. Disponible en: <https://doi.org/10.3390/act12060253>
- [23] COLLEWET, Christophe y MARCHAND, Eric. Photometric visual servoing. *IEEE Transactions on Robotics*. Agosto 2011, **27**(4), 828-840. ISSN 1552-3098.
- [24] LEVINE, Sergey, et al. End-to-end training of deep visuomotor policies. *The Journal of Machine Learning Research*. Enero 2016, **17**(1), 1334-1373. ISSN 1532-4435.
- [25] HASHIMOTO, Koichi. *Visual Servoing: Real-Time Control of Robot Manipulators via Visual Feedback*. Singapore: World Scientific Publishing, 1993. Vol. 7. ISBN 978-981-02-1507-4.
- [26] FRANKA ROBOTICS. *Franka Research 3* [en línea]. [Consulta: 19 enero 2026]. Disponible en: <https://franka.de/franka-research-3-arm>
- [27] FRANKA ROBOTICS. *Robot specifications*. En: *Franka Control Interface (FCI) Documentation* [en línea]. [Consulta: 19 enero 2026]. Disponible en: https://frankarobotics.github.io/docs/robot_specifications.html
- [28] INTEL. *Stereo Depth Camera D405* [en línea]. [Consulta: 19 enero 2026]. Disponible en: <https://www.realsenseai.com/products/stereo-depth-camera-d405/>
- [29] PROPHESSEE. *Metavision Evaluation Kit 4 - HD (EVK4)* [en línea]. [Consulta: 19 enero 2026]. Disponible en: <https://www.prophessee.ai/event-camera-evk4/>
- [30] FRANKA ROBOTICS. *libfranka 0.20.3: C++ library documentation* [en línea]. [Consulta: 19 enero 2026]. Disponible en: <https://frankarobotics.github.io/libfranka/0.20.3/>
- [31] NVIDIA. *Isaac Sim: Robotics Simulation and Synthetic Data Generation* [en línea]. [Consulta: 19 enero 2026]. Disponible en: <https://developer.nvidia.com/isaac/sim>
- [32] MAKOVYCHUK, Viktor, WAWRZYNIAK, Lukasz, GUO, Yunrong, LU, Michelle, STOREY, Kier, MACKLIN, Miles, HOELLER, David, RUDIN, Nikita, ALLSHIRE, Arthur, HANDA, Ankur y STATE, Gavriel. *Isaac Gym: High Performance GPU-Based Physics Simulation For Robot Learning*. arXiv:2108.10470 [cs.RO]. 2021. Disponible en: <https://arxiv.org/abs/2108.10470>
- [33] MACENSKI, Steven, FOOTE, Tully, GERKEY, Brian, CLAY, Chris y WOODALL, William. Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*. Mayo 2022, **7**(66), eabm6074. ISSN 2470-9476. Disponible en: <https://doi.org/10.1126/scirobotics.abm6074>
- [34] RAFFIN, Antonin, HILL, Ashley, GLEAVE, Adam, KANERVISTO, Anssi, ERNESTUS, Maximilian y DORMANN, Noah. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research* [en línea]. 2021, **22**(268), 1-8. [Consulta: 20 enero 2026]. ISSN 1533-7928. Disponible en: <http://jmlr.org/papers/v22/20-1364.html>
- [35] MARCHAND, Éric, SPINDLER, Fabien y CHAUMETTE, François. ViSP for visual servoing: a generic software platform with a wide class of robot control skills. *IEEE Robotics & Automation Magazine*. Diciembre 2005, **12**(4), 40-52. ISSN 1070-9932.
- [36] BARRIGÓN GÓMEZ, Gabriel. *Segmentación de movimiento en cámaras de eventos usando redes neuronales*

- de impulsos*. Trabajo Fin de Grado [en línea]. Santiago de Compostela: Universidad de Santiago de Compostela, 2026. [Consulta: 21 enero 2026]. Disponible en: <https://www.linkedin.com/in/gabriel-barrig%C3%B3n-g%C3%B3mez-4b8640311/>
- [37] SICILIANO, Bruno, SCIAVICCO, Lorenzo, VILLANI, Luigi y ORIOLO, Giuseppe. *Robotics: Modelling, Planning and Control*. Londres: Springer, 2009, pp. 363-407. ISBN 978-1-84628-641-4.
 - [38] FUJIMOTO, Scott, VAN HOOFF, Herke y MEGER, David. *Addressing Function Approximation Error in Actor-Critic Methods*. En: *Proceedings of the 35th International Conference on Machine Learning (ICML)*. Estocolmo: PMLR, 2018, vol. 80, pp. 1587-1596.
 - [39] HAARNOJA, Tuomas, ZHOU, Aurick, ABBEEL, Pieter y LEVINE, Sergey. *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*. En: *Proceedings of the 35th International Conference on Machine Learning (ICML)*. Estocolmo: PMLR, 2018, vol. 80, pp. 1861-1870.
 - [40] HAARNOJA, Tuomas, et al. *Soft Actor-Critic Algorithms and Applications*. En: *arXiv preprint arXiv:1812.05905*. 2018.
 - [41] ABUSADEH, Mohammed. *Navigating Soft Actor-Critic Reinforcement Learning* [en línea]. Towards Data Science, 12 de diciembre de 2024. [Consulta: 26 de enero de 2026]. Disponible en: <https://towardsdatascience.com/navigating-soft-actor-critic-reinforcement-learning-8e1a7406ce48/>
 - [42] BOCHKOVSKIY, Alexey, WANG, Chien-Yao y LIAO, Hong-Yuan Mark. *YOLOv4: Optimal Speed and Accuracy of Object Detection*. En: *arXiv preprint arXiv:2004.10934*. 2020.
 - [43] SKALSKI, Piotr. *Make Sense* [en línea]. Herramienta de etiquetado de imágenes. [Consulta: 26 de enero de 2026]. Disponible en: <https://www.makesense.ai/>
 - [44] MDN WEB DOCS. *Transmission Control Protocol (TCP)* [en línea]. [Consulta: 26 de enero de 2026]. Disponible en: <https://developer.mozilla.org/es/docs/Glossary/TCP>
 - [45] TENSORFLOW. *TensorBoard: TensorFlow's visualization toolkit* [Software]. Google. [Consulta: 26 de enero de 2026]. Disponible en: <https://www.tensorflow.org/tensorboard>
 - [46] LABBÉ, Yann, et al. *MegaPose: 6D Pose Estimation of Novel Objects via Render & Compare*. En: *Proceedings of the 6th Conference on Robot Learning (CoRL)*. Auckland: PMLR, 2022, vol. 205, pp. 488-498.
 - [47] IEEE ROBOTICS AND AUTOMATION SOCIETY (IEEE RAS). *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* [en línea]. Información general de la conferencia. [Consulta: 28 de enero de 2026]. Disponible en: <https://www.ieee-ras.org/conferences-workshops/fully-sponsored/iros>